

Master slave Flip-flop

The race around condition

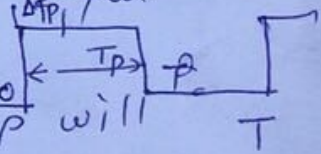
The condition $S=R=1$, was not allowed in S-R FF. This is eliminated in the J-K FF by using feedback connection from o/p. to the J & K inputs & hence we avoid the undefined state of S-R flipflop. Because of feedback connection $Q(\bar{Q})$ at the J & K of J & K, the input will change during the clock pulse ($clk=1$) if the o/p changes the state.

For example $J=K=1$, $clk=0$ $Q=0$ when a clock pulse of width t_p is applied the o/p will change from 0 to 1 after Δt is the propagation time delay. Through two NAND gates. So now after

Δt $J=K=1$ $Q=1$

Thus after Δt interval, o/p will become 0. Hence the o/p will oscillate back & forth between 0 & 1 in duration t_p of the clock pulse width. So

at the end of the clock pulse the value of Q is ambiguous. This situation is known as race around condition.



Introduction of Counters — A counter is a device which stores the number of times a particular event or process has occurred, in the form of clock pulse.

Counters can be formed by connecting individual FF together. On application of pulse, the FF in the counter undergo a change of state in such a manner that the binary number stored in the FF represent the no. of pulse applied at STP.

When clock pulses are applied to a counter, the counter progress from one state to another & the final output of the FF in the counter indicates the pulse count.

On the basis of pulse applied counter can be two types

Types of FF

Asynchronous (Ripple) Counter
(Serial)

Synchronous (Parallel) Counter

Difference between synchronous and asynchronous counters.

5. NO

Synchronous counters

Asynchronous counters

1. All ff are triggered 1. Different ff are triggered with same clock with different delay simultaneously.
2. faster in operation 3. Slower in operation
3. They are ~~also~~ known as they are known as serial counters
4. Design & implementation 4. Design & implementation are very simple.
5. The settling time of 5. The settling time is comparatively more of individual ff s.

Steps involved to design counter

1. Determine No. of FF
2. choose Type of FF
3. ~~work~~ write Truth Table for the counter
4. Derive the reset logic by K-map.
5. Draw Logic diagram
6. Draw ~~clock~~ timing diagram

1. No. of FF is determined by the modulus/mod of the counter.
Convert the modulus into binary number e.g. mod 3 means ~~100~~ (011) means 3 bit & as we know ~~an~~ a ff can store one bit, so we need three FF.

difference between synchronous and asynchronous counters.

Synchronous counters

3. NO

1. All FF are triggered with same clock simultaneously.

2. faster in operation

3. They are ~~too~~ known as parallel counters

4. Design & implementation is tedious in n bits or 10 or 100 increase.

5. The settling time of synchronous counter is highest equal to 10 of all settling time of FF.

Asynchronous counters
different FF are triggered with different clock not simultaneously.

3. slower in operation

2. They are known as serial counters

4. Design & implementation are very simple.

5. The settling time is cumulative sum of individual FFs.

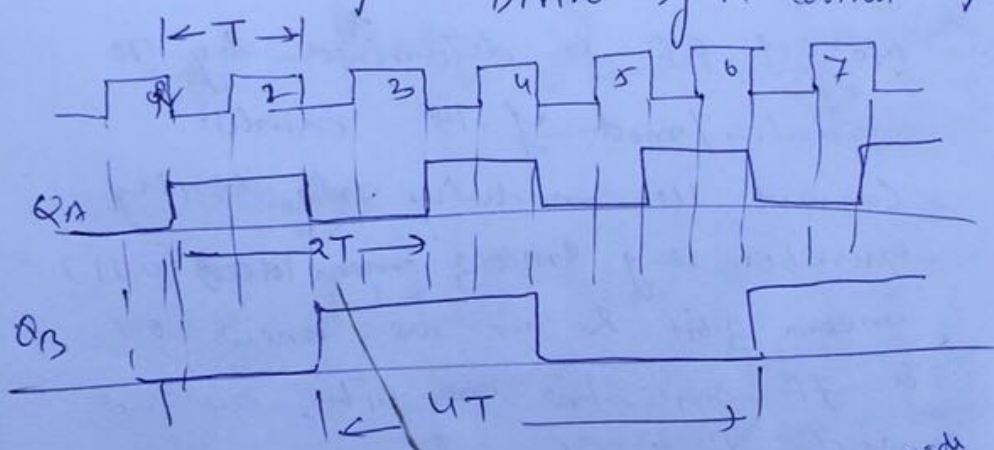
Mod number or modulus :- The mode number (modulus) of a counter is the total number of states it sequences through in each complete cycle.

$$\text{MOD-number} = 2^n$$

where $n \Rightarrow$ no. of FF

The maximum binary number counted by the counter is $2^n - 1$. Thus a 4FF counter can count as high as $= 2^4 - 1 = 15$

frequency Division or frequency Scaling
Divide by N counter



freq. of clock (f_0) = $\frac{1}{T}$

freq. of Q_A (f_1) = $\frac{1}{2T}$ } or

$$\begin{aligned} f_0 &= f_1 \\ \frac{1}{T} &= \frac{1}{2T} \\ 2f_0 &= f_1 \end{aligned}$$

$$\frac{1}{T} = \frac{1}{2T}$$

$$\frac{f_0}{f_1} = \frac{\frac{1}{T}}{\frac{1}{2T}} \Rightarrow \frac{1}{T} \times 2T$$

$$f_0 = 2f_1$$

$$\begin{aligned} f_1 &= \frac{1}{2T} \\ f_1 &= \frac{1}{2} \cdot f_0 \end{aligned}$$

Divide

Steps involved to design counter

1. Determine No. of FF
2. choose type of FF
3. ~~write~~ Truth Table for the counts
4. Derive the reset logic by K-map.
5. Draw Logic diagram
6. Draw timing diagram

1. No. of FF is determined by 110 modulus/mod of 110 counts.

Convert the modulus into binary numbers e.g. $(110)_{10} \text{ mod } 3 \text{ mean } (110)_2 (011)_2$ mean 3 bit & as we know ~~as~~ a ff can store one bit, so we need three FF

MOD N Ripple Counter

One bit counter is mod two counter

$$\text{Mod-Number} = 2^n$$

$$\text{if } n=1$$

$$\text{Mod number} = 2^1 \Rightarrow 2$$

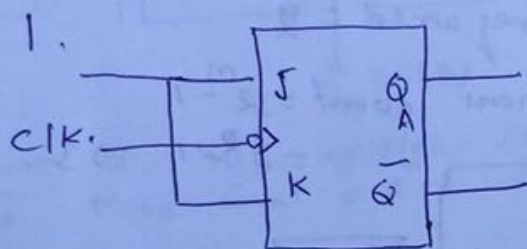
Possible count on 2 clock pulses

The max. binary number counted by 1 bit counter $= 2^n - 1$

$$= 2^1 - 1$$

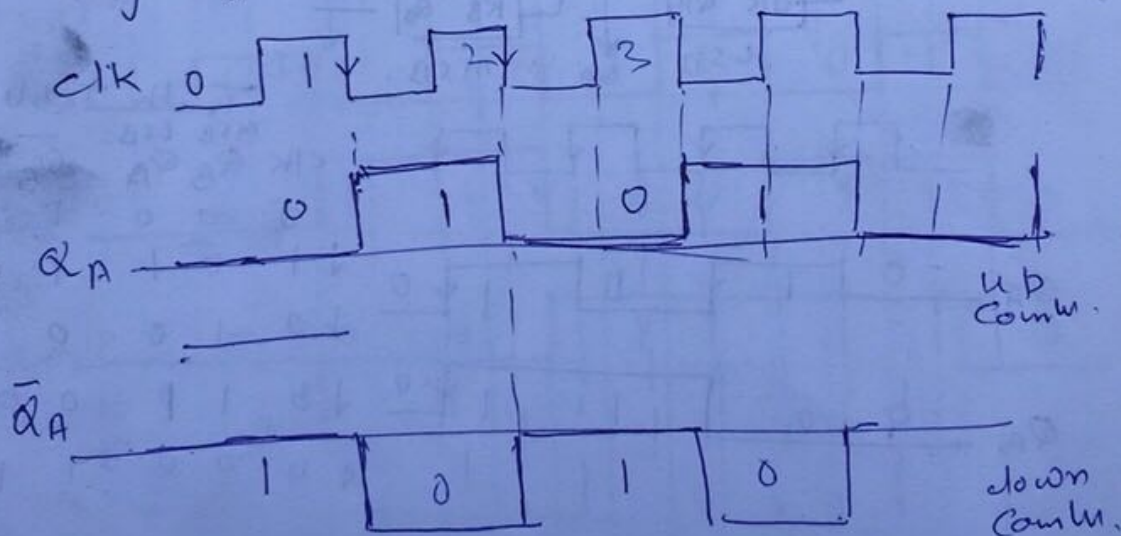
$$= 2 - 1 \Rightarrow 1$$

So possible states will be (0, 1)



Initially CLK is 0
so Q_A & $\bar{Q}_A$ will be 0. The moment we apply pulse 1 then Q will be logic 1.

Timing Diagram



Truth Table

if two clock pulse no combak to zero, so this will be two state

clk	Q_A	$\bar{Q}_A$
0	0	1
↓ 1	1	0
↓ 2	0	1

up counting Down counting

- 2 Bit up Counter (MOD-4)

$$n=2$$

$$\text{No. of Count (N)} = 2^n \Rightarrow 4$$

MOD Number = 4 Counter

- Divide by 4 Counter

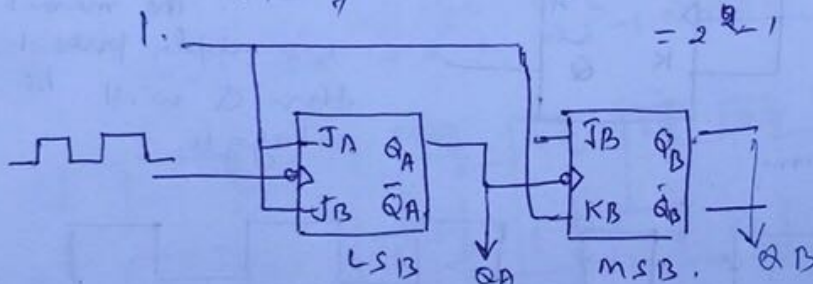
No. of ff required = 2

$$\text{No. of more count} = 2^n - 1$$

$$= 2^2 - 1 \Rightarrow 3$$

sequence

0	0
0	1
1	0
1	1



Timing diagram for a 2-bit counter (Q_A, Q_B) and its Truth Table.

Labels: L_{SB}, Q_A, M_{SB}, Q_B, Truth Table, Q_A, Q_B.

The diagram shows the following signals over 4 clock cycles:

- CLK**: A periodic square wave.
- Q_A**: The MSB, which toggles on every rising edge of the clock.
- Q_B**: The LSB, which toggles on every rising edge of the clock.

The Truth Table shows the values of Q_A and Q_B for each clock cycle, along with their complements $\overline{Q_A}$ and $\overline{Q_B}$.

CLK	Q _B	Q _A	$\overline{Q_B}$	$\overline{Q_A}$
0	0	0	1	1
↓ 1	0	1	1	0
↓ 2	1	0	0	1
↓ 3	1	1	0	0
↓ 4	0	0	1	1

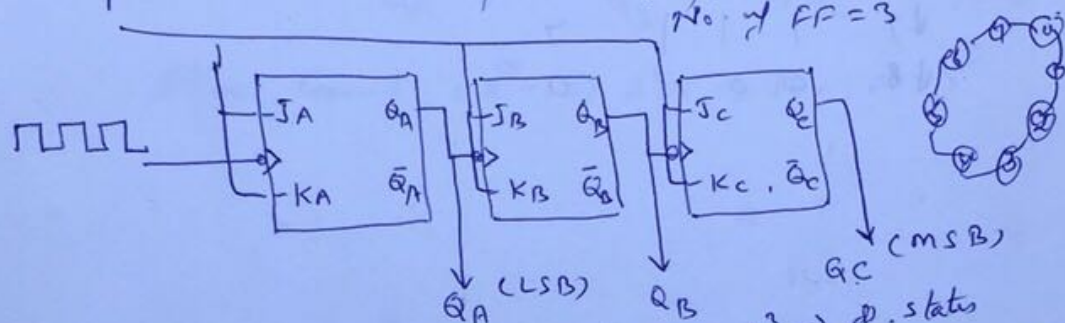
Counters further classified as

- (i) up counter
- (ii) Down Counter
- (iii) up/down counter
- (iv) Johnson
- (v) Ring Counter

3 bit asynchronous up counter

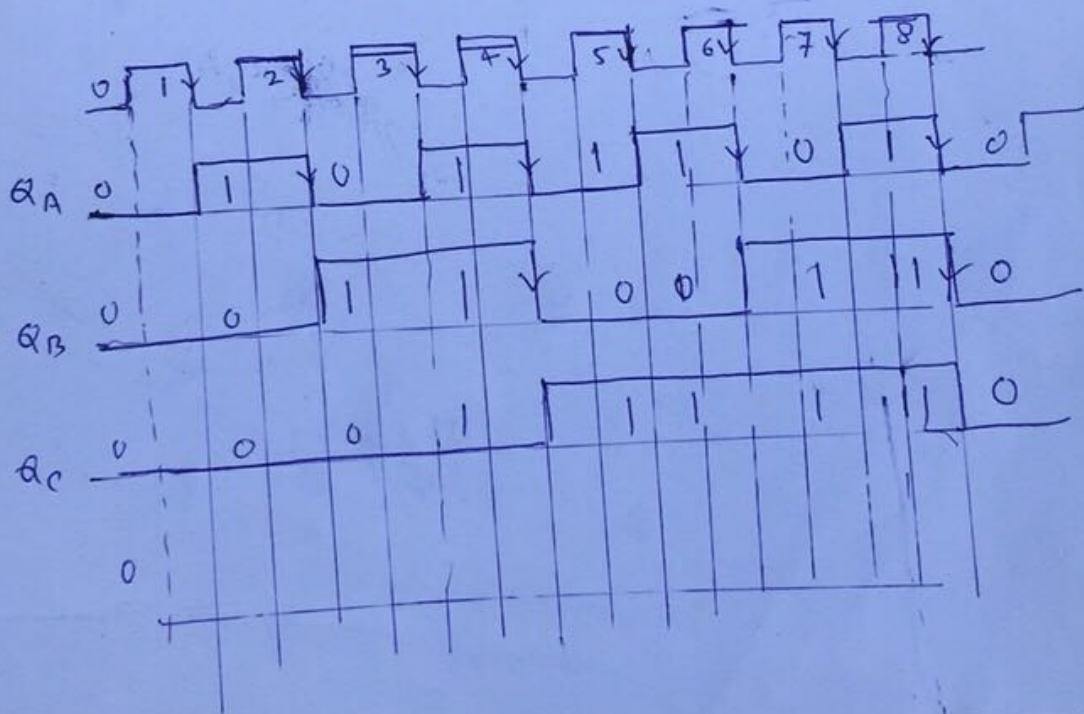
up counter counts upwards or from min to max.

No. of FF = 3



Waveform

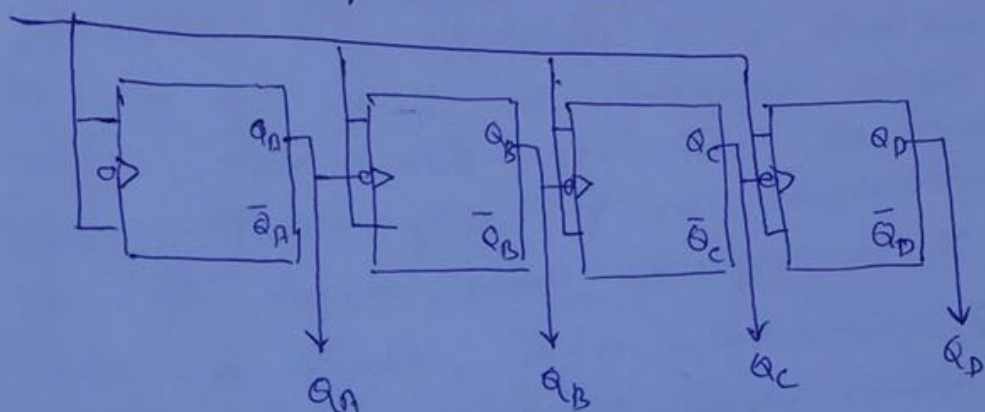
No. of states $(N)^3 = 2^3 \Rightarrow 8$ states
 Max. Count $= 2^n - 1 \Rightarrow 2^3 - 1 \Rightarrow 8 - 1 \Rightarrow 7$. (0-7)



Decimal

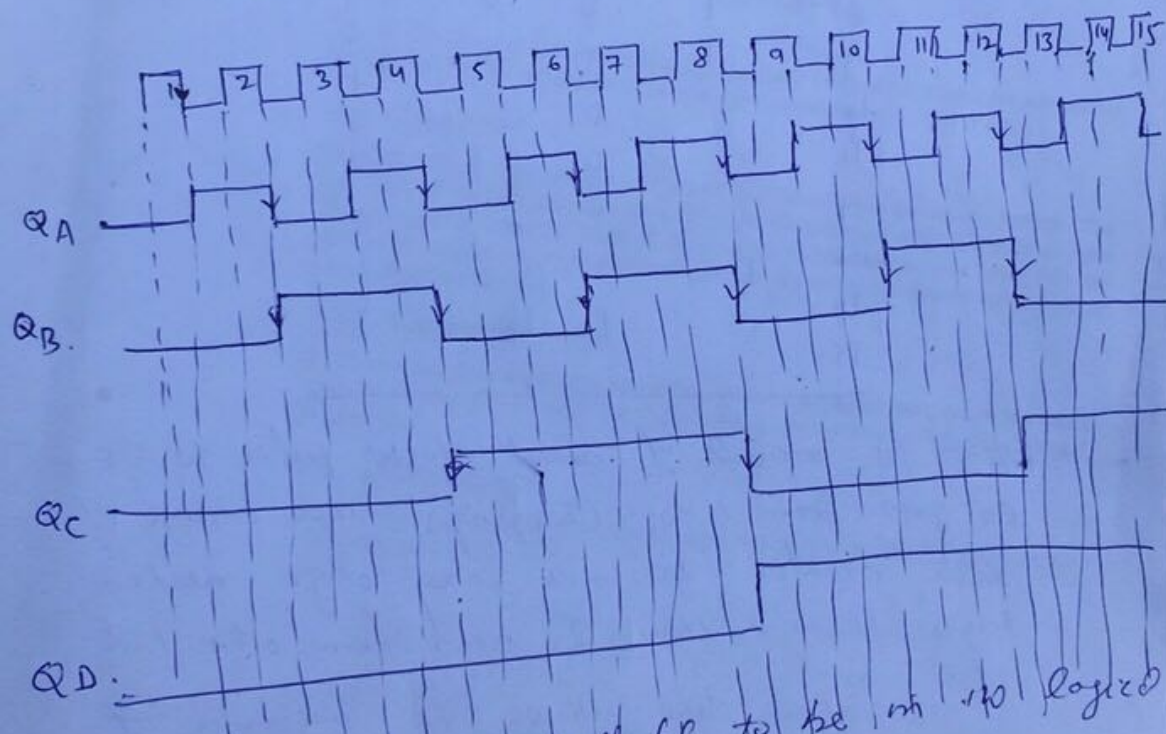
clk	state trigger	Q _C	Q _B	Q _A	
		0	0	0	0
↓ 1		0	0	1	1
↓ 2		0	1	0	2
↓ 3		0	1	1	3
↓ 4		0	0	1	4
↓ 5		1	0	1	5
↓ 6		1	1	0	6
↓ 7		1	1	1	7
↓ 8		0	0	0	0

4 bit asynchronous up counter.



$$\text{Max Count} = 2^n - 1 \Rightarrow 2^4 - 1 \Rightarrow 16 - 1 \Rightarrow 15$$

The first stage of the counter ~~has~~ switches first FF on the application of a clock pulse to the 1st ff & the successive stages change their state in turn carrying a ripple through effect of the counter pulse. As the ripple moves through the FF like a ripple, it is called a ripple counter. It is a 4-bit binary counter shown made up of JK FF.



Consider initially all FF. to be in the reset state (QA = QB = QC = QD = 0). A -ve pulse (1 to 0) which drives FF A causes Q to change logical 0 to 1. FF B does not change its state since it also requires a -ve pulse to change state B.

Table - Truth Table of 4 bit binary counter.

States	Q ₃	Q ₂	Q ₁	Q ₀
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

with 16 count of second clock pulse to FF A
 Q₃ goes from 1 to 0 (Toggling). This change of
 state creates 160 -ve going edge needed to
 trigger FF B & then Q₃ goes from 0 to 1 &
 the above table shows 160 sequence of
 binary states. An n-bit binary counter
 repeats 160 count sequence for every 2ⁿ
 (where n = # of FF) clock pulse than
 divides state for 0 to 2ⁿ - 1.

Master slave Flip-flop

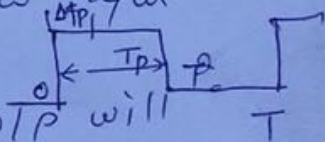
The race around condition

The condition $S=R=1$, was not allowed in S-R FF. This is eliminated in the J-K FF by using feedback connection from o/p. to the J & K inputs & hence we avoided the undefined state of S-R Flipflop. ~~During the clock pulse~~ Because of feedback connection $Q(\bar{Q})$ at the J & K inputs, the input will change during the clock pulse ($clk=1$) if the o/p changes the state.

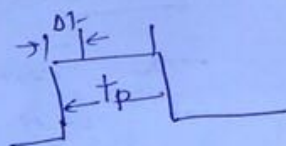
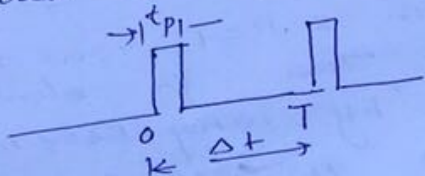
For example $J=K=1$, $clk=0$, $Q=0$ when a clock pulse of width t_p is applied the o/p will change from 0 to 1 when Δt is the propagation time delay. Through two NAND gates. So now

Δt $J=K=1$ $Q=1$

Thus after Δt interval, O/P will become 0. Hence the o/p will oscillate back & forth between 0 & 1 in duration t_p of the clock pulse width. So at the end of the clock pulse the value of Q is ambiguous. This situation is known as race around condition.



The race around condition can be avoided if $t_p < \Delta t$



This race around condition can be avoided by Master Slave Flip Flop

Introduction of Counters - A counter is a digital device which stores the number of times a particular event or process has occurred, in the form of clock pulse.

Counters can be formed by connecting individual FF together. On application of pulse, the FF in the counter undergo a change of state in such a manner that the binary number stored in the FF represents the no. of pulse applied at STP.

When clock pulses are applied to a counter, the counter progresses from one state to another & the final state of the FF in the counter.

Types of CF

On the basis of pulse applied, counter can be

Types Asynchronous Counter
(Ripple Counter)

Synchronous Counter
(Parallel Counter)

Asynchronous (Ripple) Counter - If all the

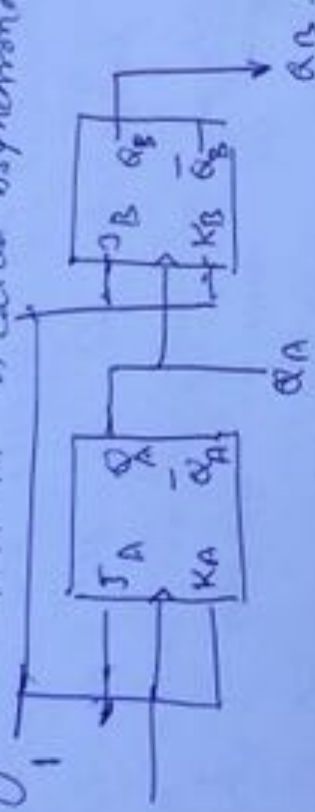
FF are not

clocked at the same time, the clock pulse is applied to the first FF

& the successive FF is triggered

by the output of the previous FF.

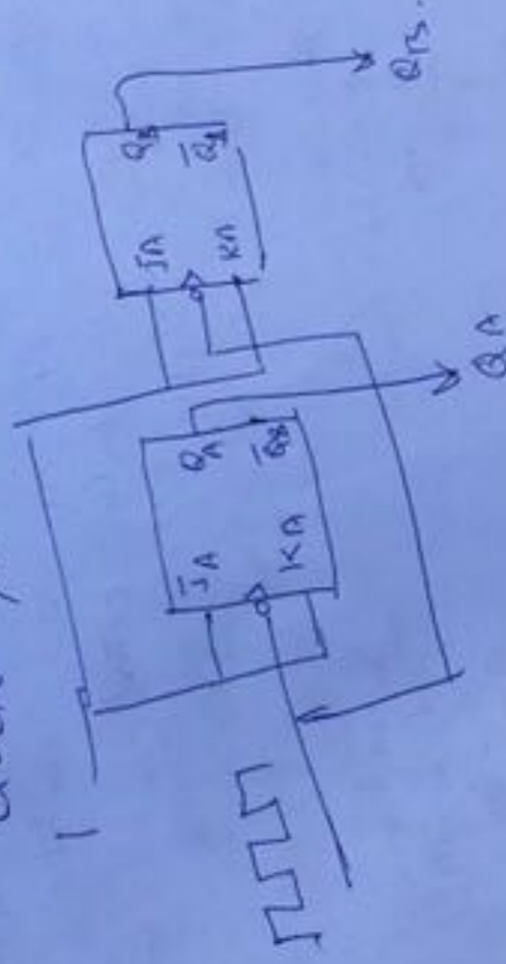
When this is called asynchronous counter



Synchronous (Parallel) Counter :- If all the FF are

clocked/connected with same

clock pulse



ASM Chart

ASM stands for Algorithmic State Machine (ASM) also called State

Machine (SM) Chart, useful for describing detailed logic, just like flow chart.

It gives about 10 hardware required.

ASM chart describe the sequence of events as well as the timing relationships between the state of a sequential circuit.

The chart composed of three elements

1. Start Box
2. Decision Box
3. Conditional Box



represents initial module



binary decision



Conditional O/P.

when O/P is 1 & then it is used always in next stage, if it is depend on next module then it is used.

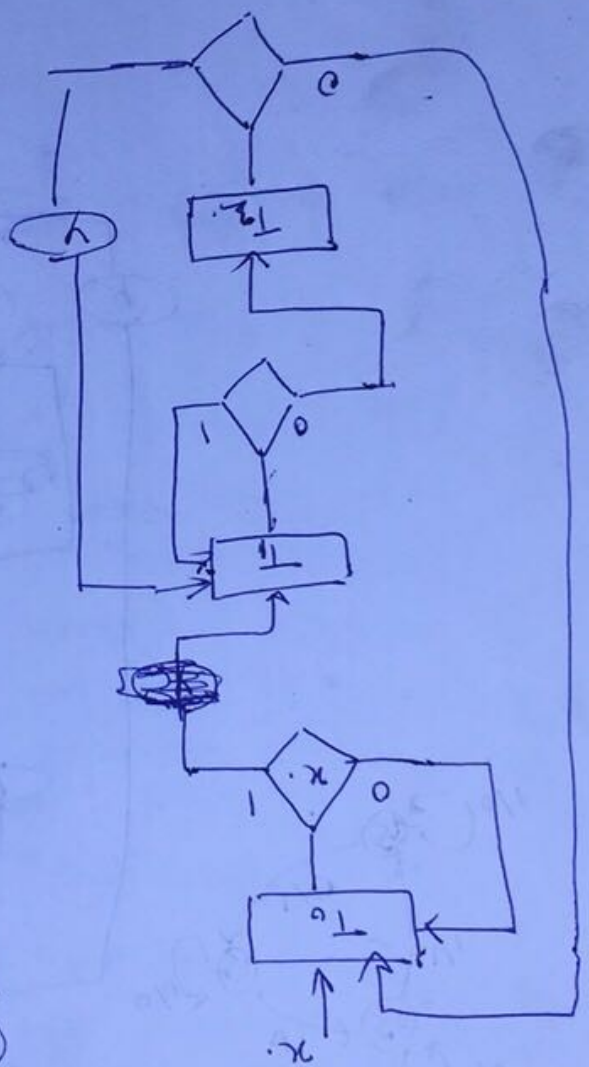
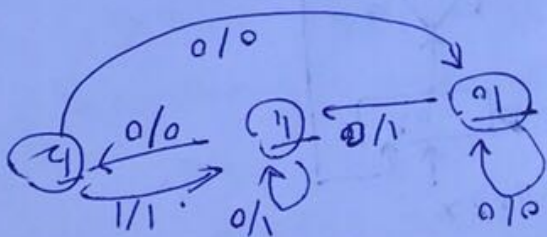


Answer

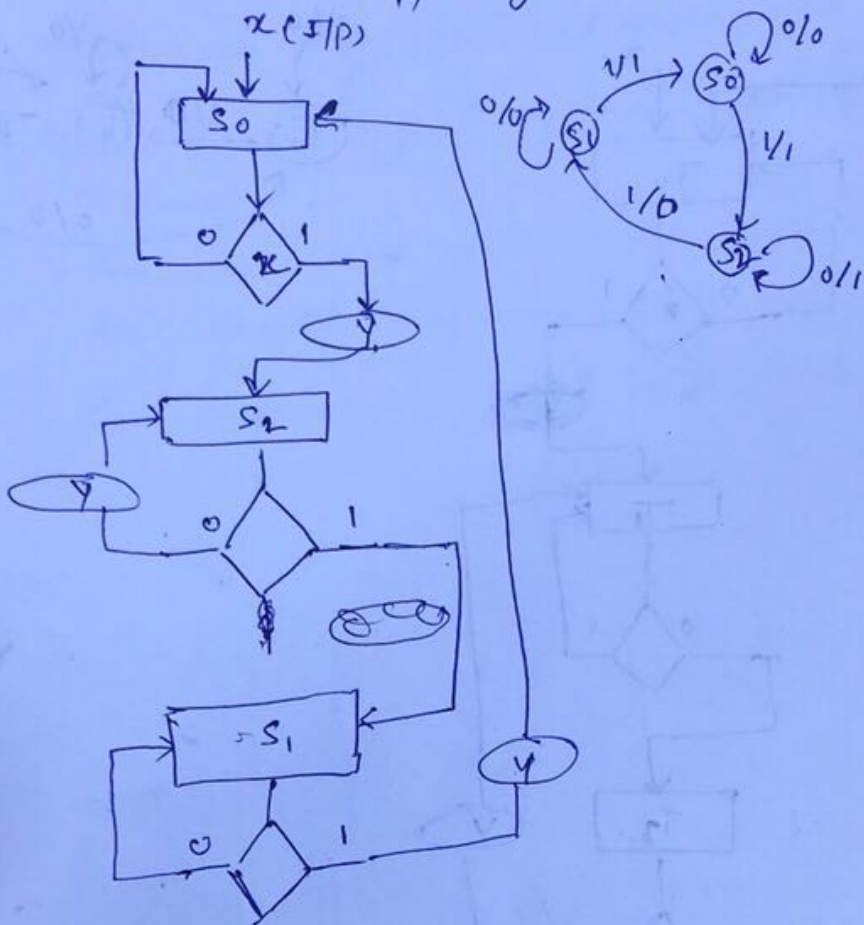
Advantages of SM stand

1. Refreshed physical hard ware
2. Operation of digital system can be easily unsolved
3. Can be used both combinational as well sequential etc
4. Can compressed stack of P table.

Draw the Arm chart of given slot Design

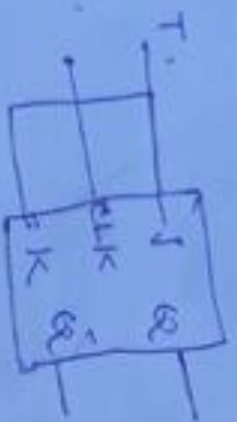
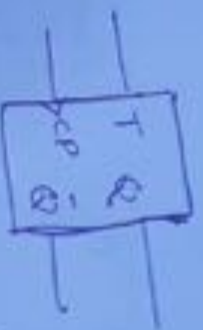


Prob Draw the ASM chart of the given state diagram of Mealy M/C.



T-Flip Flop

T FF is much simpler version of J-K FF. Both J & K inputs are connected together & has one 5/P which is named T. T stands for toggle.



Graphical Symbol.

when $T=0$ state ($J=K=0$) prior to a clock pulse, the Q & Q/P will not change. when $T=1$ ($J=K=1$) prior to a clock, the Q/P will be in the Q state after clocking. If $T=1$, $clk=1$ the Q/P state will change state regardless of what Q/P was prior to clocking. This is called toggle.

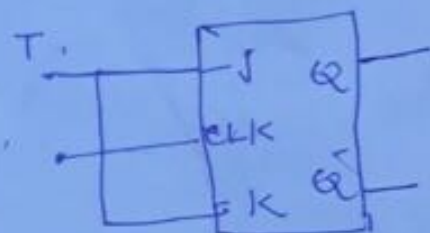
Truth Table

Qn	T	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

The Truth table shows that when $T=0$ then $Q_{n+1} = Q_n$ i.e. the next state is same as the present state when $T=1$ the $Q_{n+1} = Q_n$ i.e. FF is complemented.

T-Flip Flop

- * T FF is much simpler version of J-K FF.
- * Both the J & K inputs are connected together & has one I/P which is known as T I/P - T stands for toggle.



Graphical symbol.

when $T=0$ state ($J=K=0$) prior to a clock pulse, the Q o/p will not change. when $T=1$ ($J=K=1$) prior to clocking, the o/p will be in the $\bar{Q}$ state after clocking. If $T=1$, $clk=1$ the o/p state will change state regardless of what o/p was prior to clocking. This is called Toggle.

Truth Table

Q_n	T	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

The Truth table shows that when $T=0$ then $Q_{n+1} = Q_n$ i.e. the next state is same as the present state when $T=1$ the $Q_{n+1} = \bar{Q}_n$ i.e. FF is complemented.

present state when $T=1$ the $Q_{n+1} = \bar{Q}_n$ i.e. FF is complemented.

Slack diagram & state table



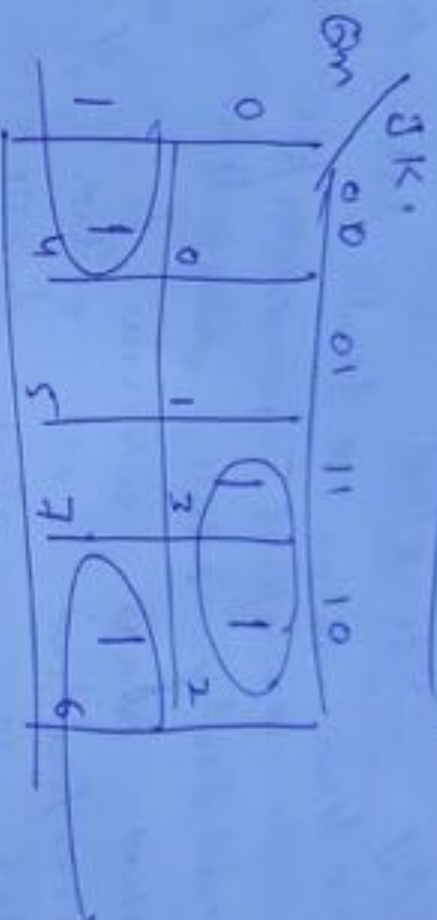
From the above diagram, we can easily understand that the state transition from 0 to 1 takes whenever T is applied ($T=1$) irrespective of K value. Similarly, state transition from 1 to 0 takes whenever K is applied ($K=1$) irrespective of T value. Also, the state transition from 0 to 0 occurs whenever $T=0$ and the state transition from 1 to 1 occurs whenever $K=0$.

Present state & Next state table

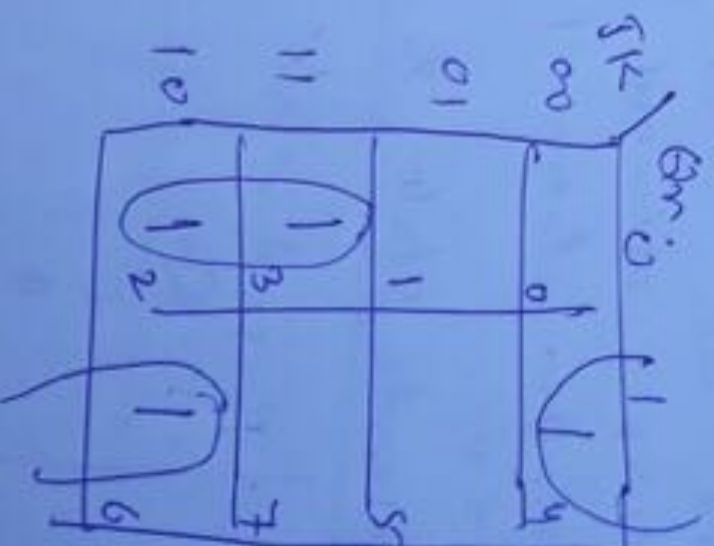
Present state Qm	T	T/P K	Next state Qn+1
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0



$$Q_{n+1} = \bar{Q}_n J + Q_n \bar{K}$$



$$Q_{n+1} = \bar{Q}_n J + Q_n K$$

②

Case - 4 $J = K = 1$ deserves special attention.

when $J = 1$, $K = 1$ & the previous state is set state ($Q_n = 1, \bar{Q}_n = 0$) then =

$$S = J\bar{Q}_n = 1 \cdot 0 = 0$$

$$R = KQ_n = 1 \cdot 1 = 1$$

So $S = 0$ & $R = 1$, the FF Reset on the application of a clock pulse i.e. the FF will toggle from set to Reset state.

Again for $J = K = 1$ if the previous state is a Reset ($Q_n = 0, \bar{Q}_n = 1$)

$$S = J\bar{Q}_n = 1 \cdot 1 = 1$$

$$R = KQ_n = 1 \cdot 0 = 0$$

$$\text{Since } S = 1, R = 0$$

the flip flop on the application of clock pulse will toggle to the set state. Hence when $J = K = 1$, the FF toggles on the application of next clock pulse.

Truth Table of J-K Flip

CLK	J/P		Q/P		Remark
	J	K	Q _{n+1}	Q _n	
X	0	0	Q _n	Q _n	No change.
1	0	1	0	0	Reset
1	1	0	1	1	Set
1	1	1	$\bar{Q}_n$	$\bar{Q}_n$	Toggle

It is important to realise
that because of the feedback
connection in the JK FF, a clock
pulse that remains in the 0 state
while $J=K=1$ will cause the output
to complement again & again until
the clock pulse again goes to 0.

To avoid this undesirable operation,
the clock pulse must have a
limited duration that is shorter
than propagation delay time of the FF.

From the state diagram, we can draw the present state table and the next state table - Non-stable D-input

Present state (Q_n)	D-input	Next state (Q_{n+1})
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Table

Q_n	Q_{n+1}	Input (D)
0	0	0
0	1	1
1	0	0
1	1	1

Characteristic Equation

$$Q_{n+1} = D$$



or



$$Q_{n+1} = D$$

Hence in a D-FF, the next state follows the Delay SIP

J-K Flip Flop

The characteristics similar

* A J-K FF has the characteristics similar to S-R FF.

* It has an input of an S-R FF. In SR FF, if

* It can be defined as modification of SR FF.

* In addition, the undefined state of

* SR is permitted in it.

* Input J & K have to be 0 like J/P S/R

for set & Reset FF respectively. J & K

when J = K = 1, the FF toggles

i.e. switches to its complement state

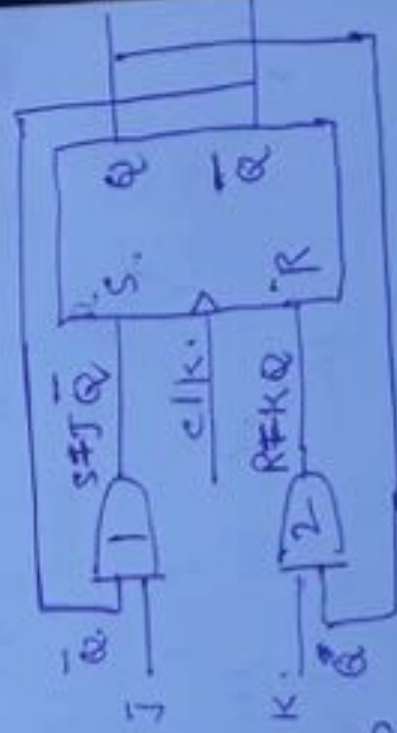
if $Q = 0$ then $\bar{Q} = 1$ & vice versa

* A J-K FF can be constructed from

the clocked S-R FF with the help

of two AND gates as shown in

the fig.



* The data J & K are connected to the

first gate & its output (JQ) is applied to

the S input of S-R flip-flop

* Similarly the data J & K are connected

to second AND gate & its output KQ is

applied to R input of S-R flip-flop

WORKING OF J-K FF

When $J=0$ and $K=0$, the FF will stay in its previous state. When $J=0$ and $K=1$, the FF will reset to 0. When $J=1$ and $K=0$, the FF will set to 1. When $J=1$ and $K=1$, the FF will toggle its state.

Case-1: When $J=0$ and $K=0$, the FF will stay in its previous state. When $J=0$ and $K=1$, the FF will reset to 0. When $J=1$ and $K=0$, the FF will set to 1. When $J=1$ and $K=1$, the FF will toggle its state.

Case-2: When $J=0$ and $K=0$, the FF will stay in its previous state. When $J=0$ and $K=1$, the FF will reset to 0. When $J=1$ and $K=0$, the FF will set to 1. When $J=1$ and $K=1$, the FF will toggle its state.

Case-3: When $J=1$ and $K=0$, the FF will set to 1. When $J=0$ and $K=1$, the FF will reset to 0. When $J=1$ and $K=1$, the FF will toggle its state.

When $J=1$ and $K=1$, the FF will toggle its state. This is because the J and K inputs are both 1, which causes the FF to toggle its state on each clock pulse.

Case-4: When $J=1$ and $K=0$, the FF will set to 1. When $J=0$ and $K=1$, the FF will reset to 0. When $J=1$ and $K=1$, the FF will toggle its state.

When $J=1$ and $K=1$, the FF will toggle its state. This is because the J and K inputs are both 1, which causes the FF to toggle its state on each clock pulse.

①

D Flip Flop

* This D FF has only one I/P called

Delay(D)

& two O/P Q & $\bar{Q}$.

* It can be constructed from an S-R

FF by inserting an inverter between

S & R and assigning the symbols

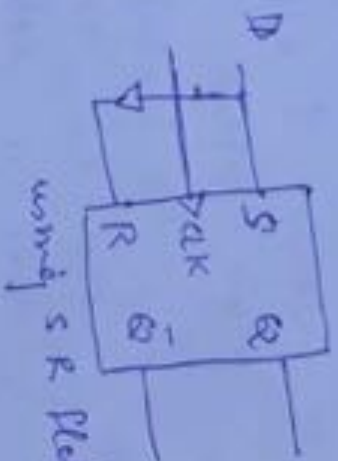
D to

input S.

no change in the

Fig -

Q	S	R	Q _{next}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



using S-R Flip Flop

logic symbol

A load I/P only
Nand gate, fixed to I/P high

Case-1

CLK = 0

D = 0

No change.

Case-2

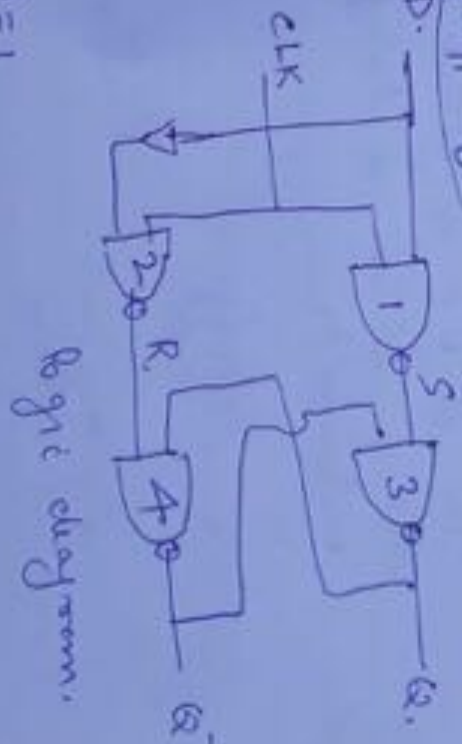
CLK = 1 D = 1

Q = 1 $\bar{Q}$ = 0

Case-3 CLK = 1 D = 0

Q = 0

$\bar{Q}$ = 1



Truth Table.

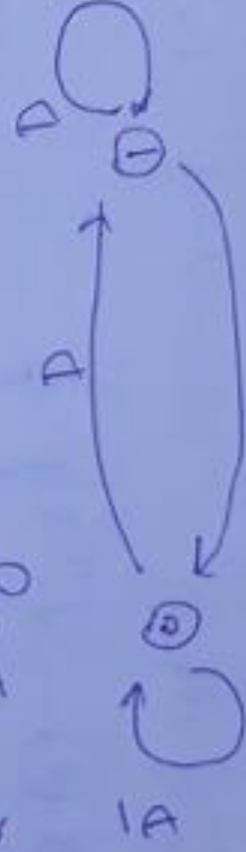
clk	D	Q _{next}
1	0	0
1	1	1
0	x	Q _{next}

It is clear that the flip flop at time (Q_{next}) follows the value of S/P D when no clock pulse is applied.

As transfer of data from the S/P to the o/p is delayed, it is known as delay (D) FF.

The D FF is either used as delay device or as a latch to store bit information.

State Diagram:



for the state diagram, it is clear that D=1, the next state will be 1. When D=0, the next state will be 0, irrespective of previous state.

(2)

NAHD Based SR tablecharacteristic. truth table of SR flip flop

	Present state S ₀	clocked clk	S	R	Next state Q _{n+1}	Remark
1	0	0	0	0	0	No change
2	0	0	0	1	1	No change
3	0	1	0	0	0	No change.
4	0	1	0	1	1	No change.
5	1	0	0	0	0	No change.
6	1	0	1	0	0	No change.
7	1	0	1	1	1	No change.
8	1	1	0	1	0	Reset-
9	0	1	0	1	0	Reset-
10	1	0	1	0	1	No change.
11	0	1	1	0	1	set
12	1	1	1	0	1	set
13	0	0	1	1	0	No change
14	0	0	1	1	1	No change
15	0	1	1	1	?	no use
16	1	1	1	1	?	

$S=0$ $R=0$ $clk=0$
The flip flop will stay in its present state

$S=0$ $R=0$ $clk=1$

Again the flip flop will stay in its present state, the first four rows indicate no change.

$S=0$ $R=1$ $clk=0$

The flip flop will remain in its present state the NAND

$S=0$ $R=1$ $clk=1$, the NAND

gate 1 o/p will go to become 1 & the

NAND gate o/p will be 0. Now a 0 at gate-4 will force $\bar{Q}=1$ which

is 1/p of gate-3 hence $A=0$

Case 3 - ~~set~~ $S=1$ $R=0$ $clk=0$ no change

$S=1$ $R=0$ $clk=1$

NAND gate 1 o/p goes to 0 & gate o/p 1

Now 0 at gate 1 o/p goes to 0 & gate 2 o/p goes to 1. Now a 0 at NAND

gate 3 STP force Q to 1 which in lines force $\bar{Q}=0$

Case-4 $S=1$ $R=1$ $clk=0$, the result is in the o/p of gate 1 & gate 2 and also 1 on both Q & $\bar{Q}$

when clk is high $clk=1$
 the values of S & R inputs will
 be passed to the o/p of AND gate
 & no o/p of CO1 if the flip flop
 will change according to the changes
 in S & R o/p.

Case 1 - $R=0$ $S=0$
 $clk=1$

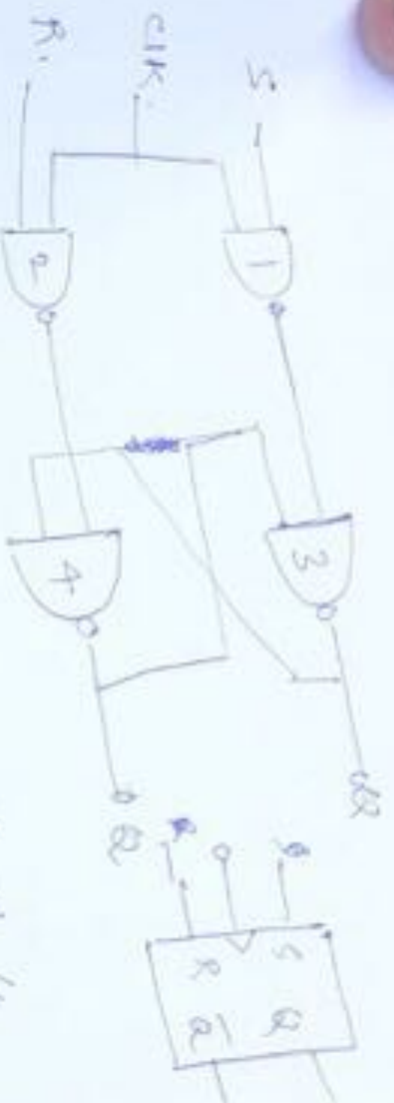
Case 2 $S=0$ $R=1$
 $clk=1$

clk	S	R	Q	
0	0	0	0	no change
1	0	0	0	no change
1	0	1	0	reset
1	0	1	1	set
1	1	1	1	undefined

Note: Apply - o/p of NOR gate is 0 if any
 o/p is 1.

NAND Based SR latch

①



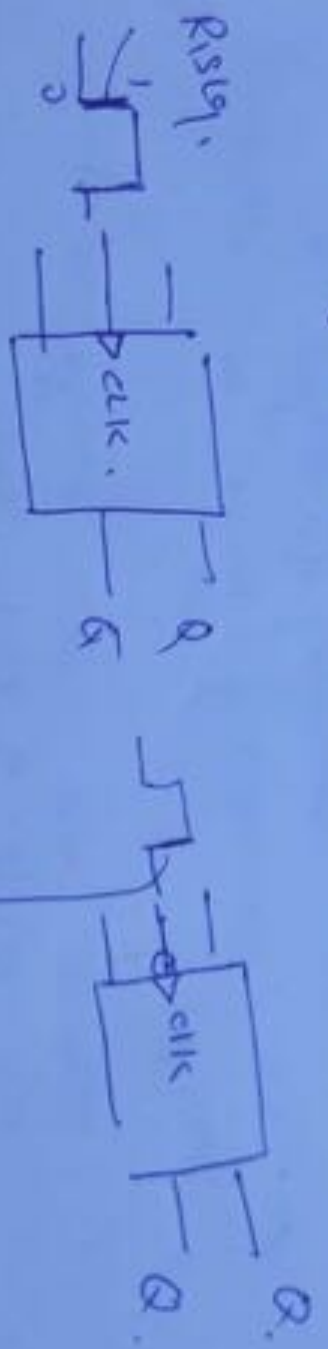
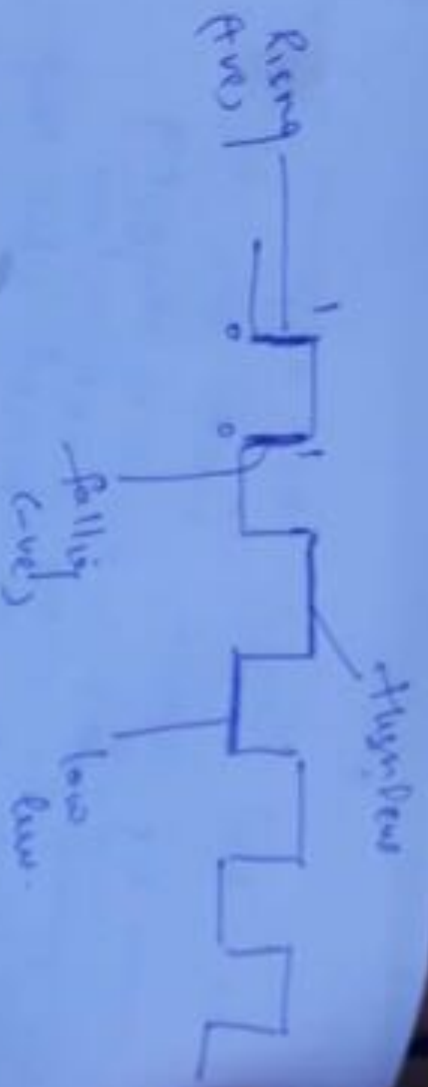
The SR flip flop circuit consists of 4 NAND gates.

The SR input control the clock. The SR flip flop in the circuit is unlocked SR latch. But the flip flop does not respond to these inputs until the rising edge of clock signal occurs.

at Power at any S/O of NAND gate will be its output high & both input high then output 'low'.

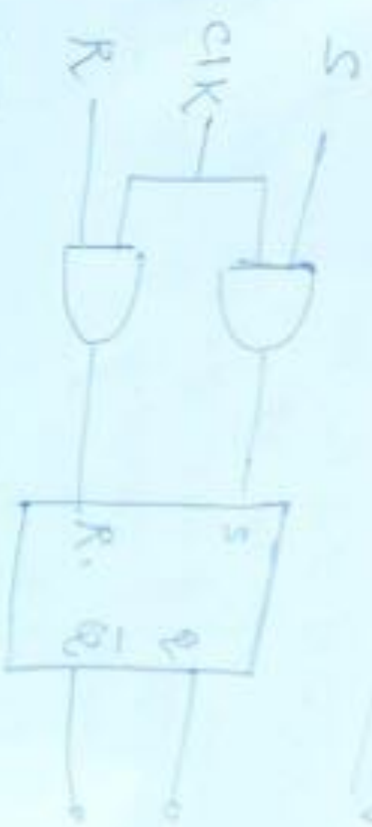
SR	Q
00	1
01	1
10	1
11	0

0 1 → 1
Q Q → 0

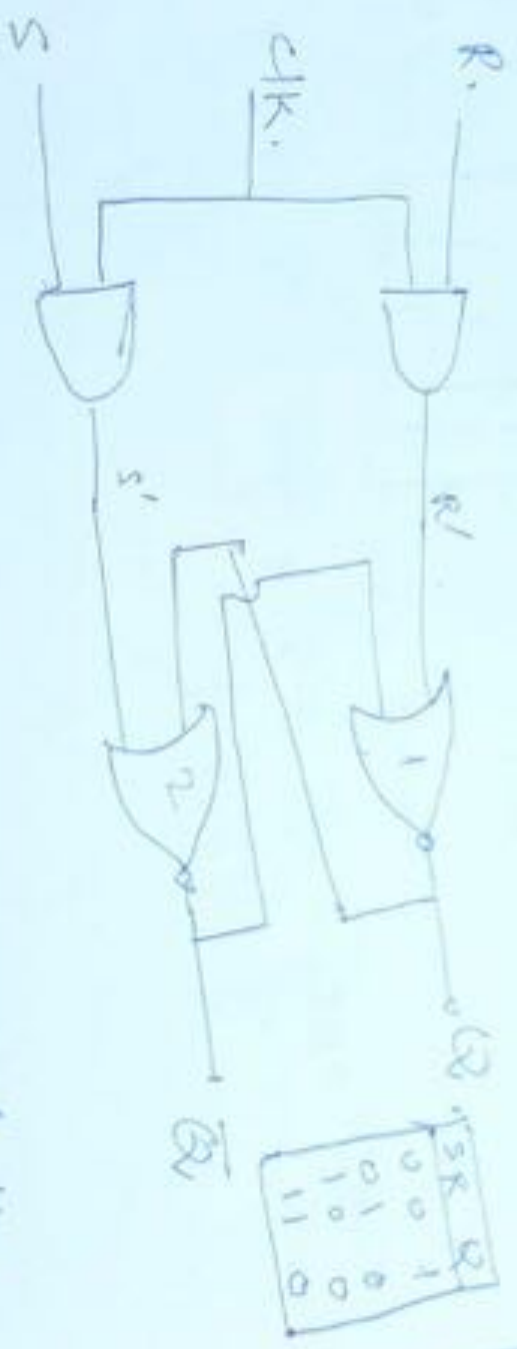


low hanging

Edge Flop clock Based SR latch



The SR flip consist of two additional AND gate and two 1 to 2 SR



S	R	Q
0	0	1
0	1	0
1	0	0
1	1	0

In the next clock $CLK=0$ the O/P of the two gates are 0 and change in the value of SR flip will not affect the O/P (Q) of the flip.

S	R	Q
0	0	0
0	1	0
1	0	0
1	1	1

latches & latches

1. This Asynchronous
2. This does not have
3. any clock
4. It is built up by logic gates
5. A latch is responsive towards the i/p switch & also competent in transmitting information on enabled when the switch is ON
6. Latches are faster
7. consumes less power
8. Latches are responsive towards faults on enable pin
9. have trigger enable

FFs.

1. This is synchronous
2. This has clock signal
3. It is made up of - latch by adding enable clock signal.
4. They are responsive towards the clock signal moreover the o/p will not vary until a change in the i/p takes place within the clock signal
5. FF are slower.
6. consumes more power
7. FF are protected towards fault
8. edge trigger

clock:

1. Synchronous logic ckt requires means by which events can be sequenced such as how in triggering rising means making a ckt active rising clock is a special ckt that and electrical pulse through a ckt

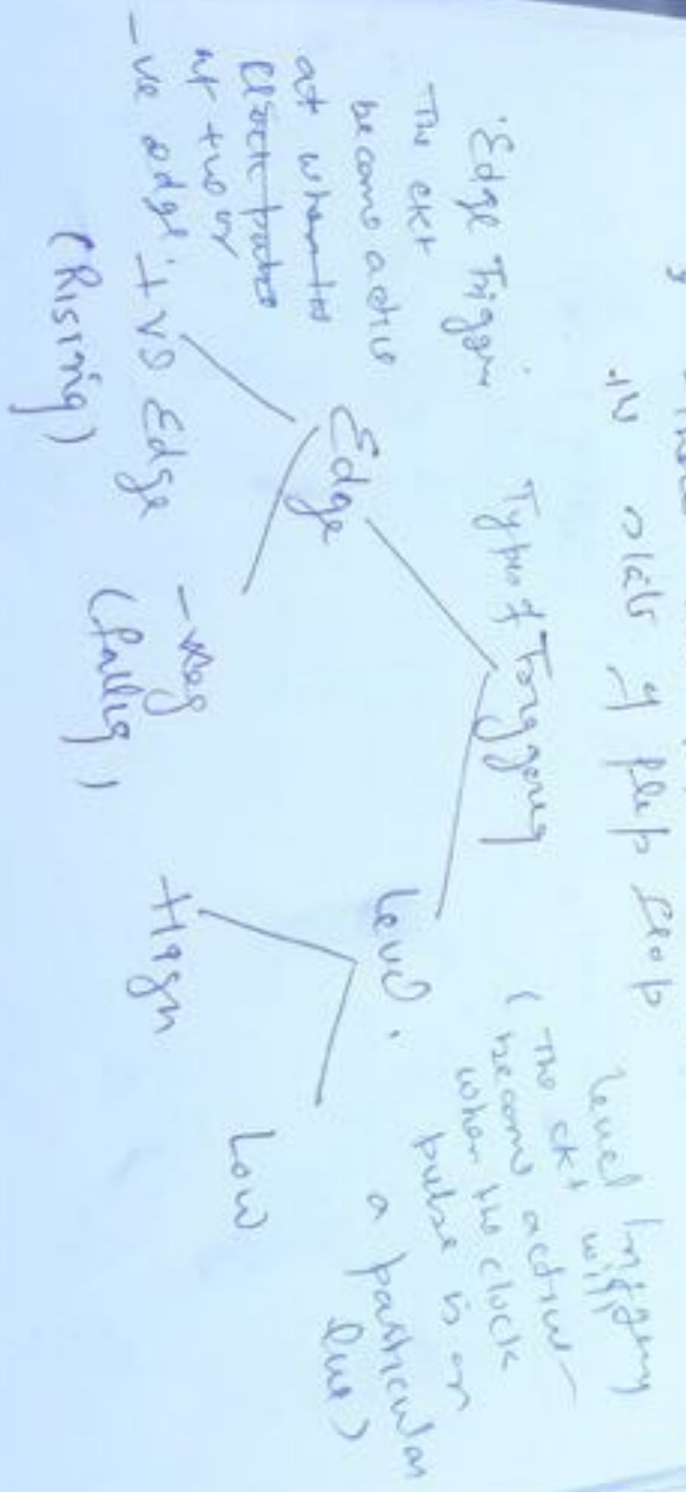


Triggering: -x- The state of flip flop is changed by memory change

in its memory if signal. This memory change is called.

Triggering:

2. There are four ways to change its state of flip flop





Present state (Q_n)	S	R	Next state (Q_{n+1})
0	0	0	0
1	0	1	0
2	0	0	1
3	0	1	1
4	1	0	0
5	1	1	0
6	1	0	1
7	1	1	d.

Excitation Table for S-R latch

An excitation table shows the minimum number of inputs required to generate a particular next state.

Q_n	Q_{n+1}	S	R
0	0	0	d
0	1	1	0
1	0	0	1

Characteristic Equation

Q_n	S	R	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	d
1	0	0	1
1	0	1	d
1	1	0	0
1	1	1	d

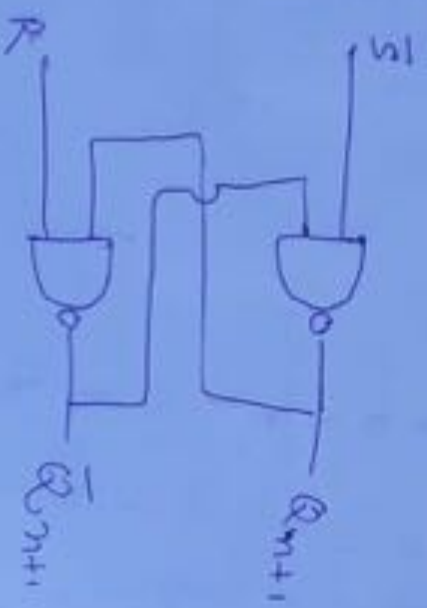
Q_n	S	R	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	d
1	0	0	1
1	0	1	d
1	1	0	0
1	1	1	d

$$Q_{n+1} = Q_n \bar{R} + S \bar{R}$$

$$= \bar{R} (Q_n + S)$$

$$Q_{n+1} = S \bar{R} + \bar{R} Q_n$$

$$= \bar{R} (S + Q_n)$$



- Case 1 $S = 0$ $R = 0$ $(Q_n, \bar{Q}_n)$
2. $S = 0$ $R = 1$
3. $S = 1$ $R = 0$
4. $S = 1$ $R = 1$

$\bar{S}$	$\bar{R}$
0	0
0	1
1	0
1	1

Q_{n+1} $\bar{Q}_{n+1}$
 undefined (a)
 0 set
 1 reset
 Q_n $\bar{Q}_n$

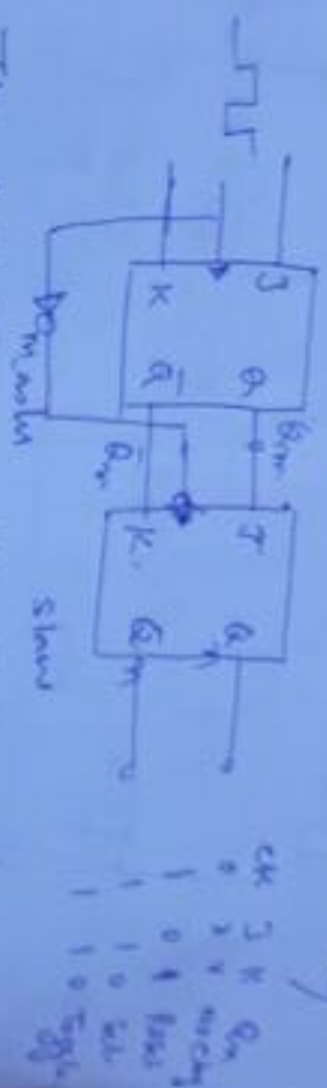
Truth table of T flip-flop

T	Q_{n+1}
0	1
0	1
0	1
0	1
1	0

To analyze the above circuit, we must remember that a NAND gate will give 0 if any input is 1.

Master Slave J-K FF

* A master slave FF can be constructed using two J-K FF as shown in fig.

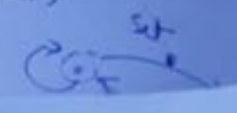


The first FF is called master & its output called slave, master FF is driven by its two inputs of its clock pulse & slave is driven by its negative edge of its clock pulse.

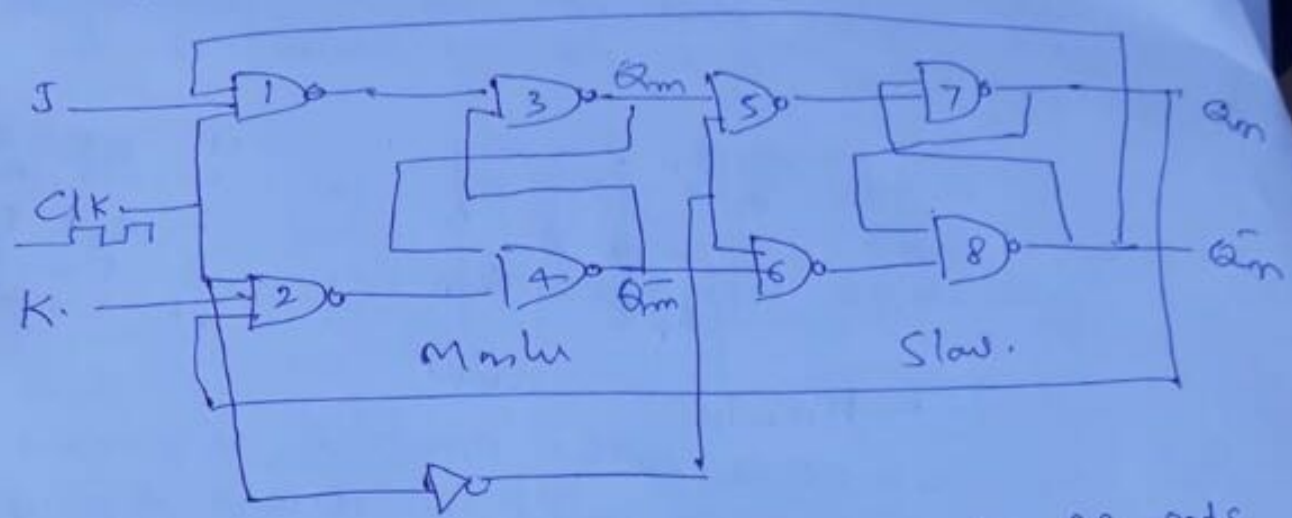
Therefore when clock has its positive pulse edge of its clock, its master acts according to its J-K FF but its slave does not respond into its requires a -ve edge of its clock FF pulse.

when clock FF has -ve edge, its slave FF copies its master's output. But master does not respond to its feedback from Q & Q-bar, since it requires a two edge of its clock FF - this master slave FF does not have race around condition.

Present state (Q)	S	R	Next state (Q _{next})
0	0	0	0
1	0	0	1



A master slave FF is constructed using NAND gates



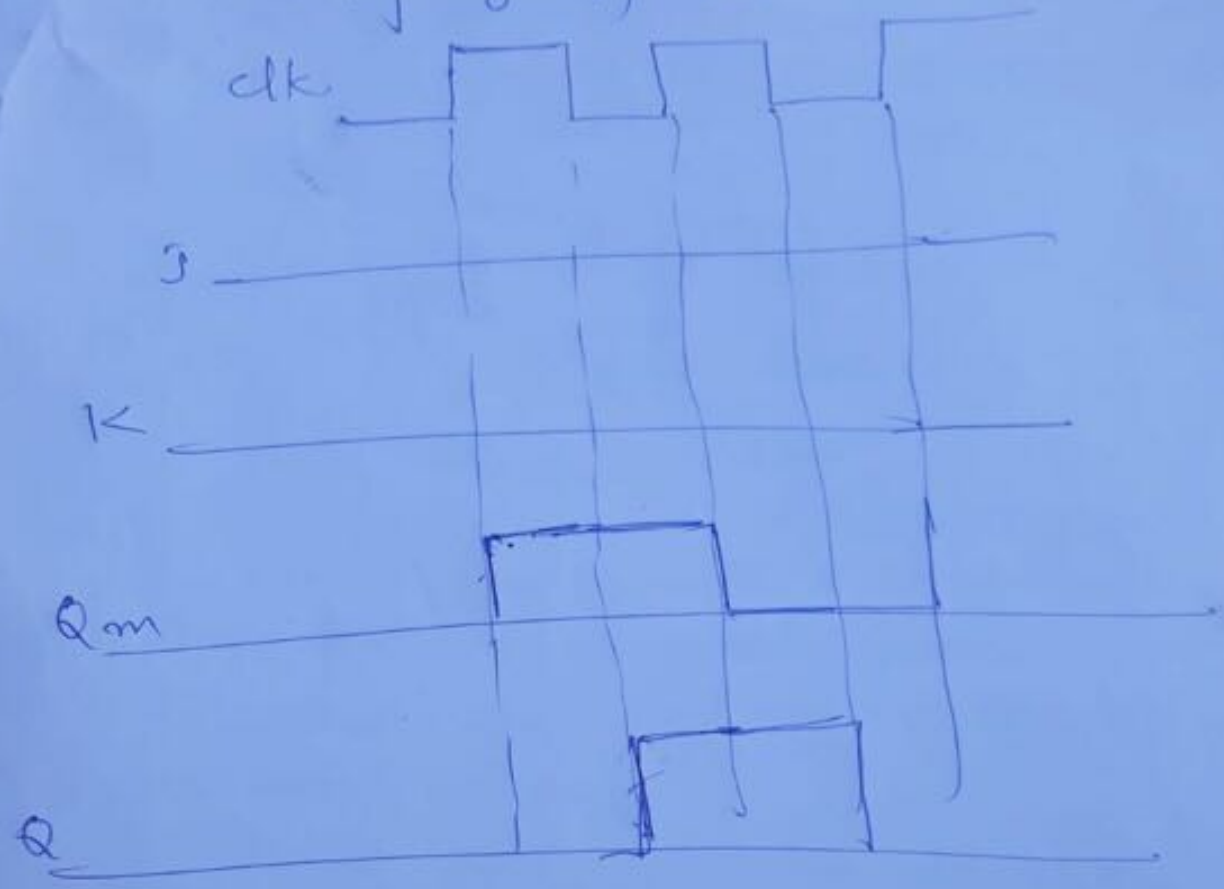
When $J=1$ $K=0$, the master ff sets on the clock edge. The high ($Q=1$) o/p of the master ff drives the $\bar{S}$ of the slave. So when the negative clock edge hits, the slave also sets. The slave ff copies the action of the master ff.

If $J=0$ $K=1$, the master resets on the edge of the clock pulse. High $\bar{Q}$ o/p of the master drives input K of the slave ff. Then the slave ff resets at the edge of the clock pulse. Once again the slave ff copies the action of the master ff.

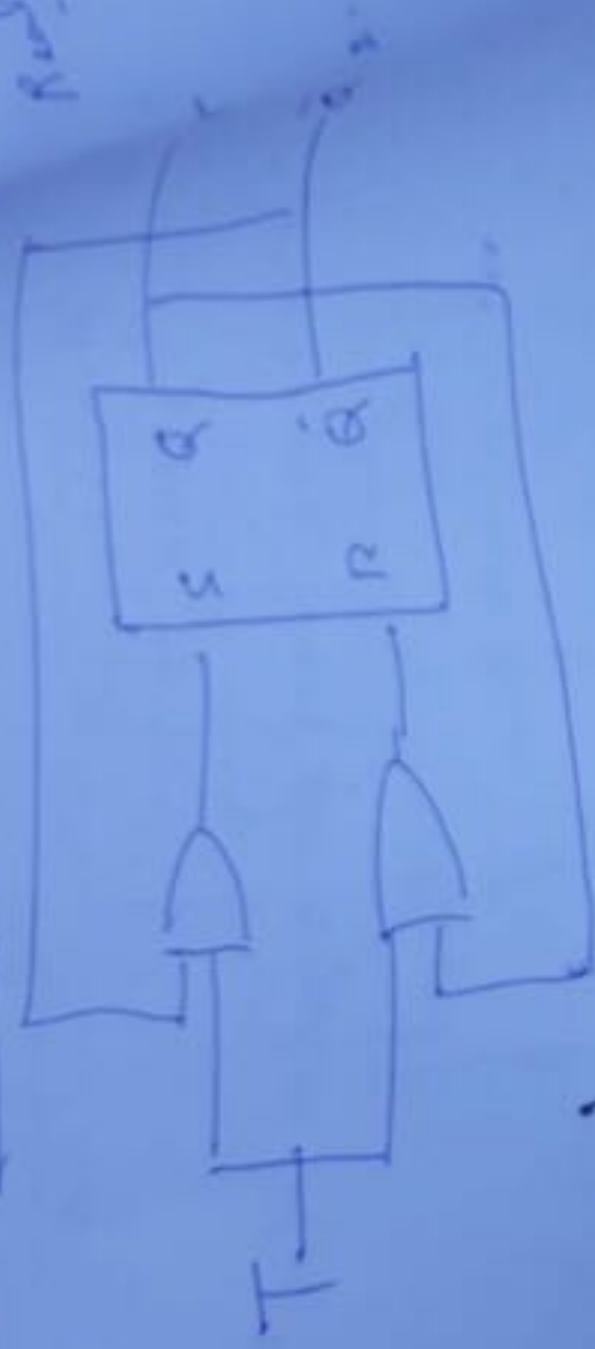
If $J=K=1$, the master ff toggles on the edge of the clock & slave on the edge of the clock. The condition $J=K=0$ is no do.

Present state (Q)	S	R	Next state (Q _{next})	Set
0	0	0	0	0

Timing diagram of Master S/R FF



5- Gr circuit Diagram



Realization of J-K FF using S-R FF

Available FF - S-R
 Required FF - J-K

2. Characteristic Table for required FF (JK)

Q _n	J	K	Q _{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

3.

Excitation Table for S-R FF

Q _n	J	K	Q _{n+1}	S	R
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	1	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	0	0	1
1	1	0	1	0	0
1	1	1	0	0	1

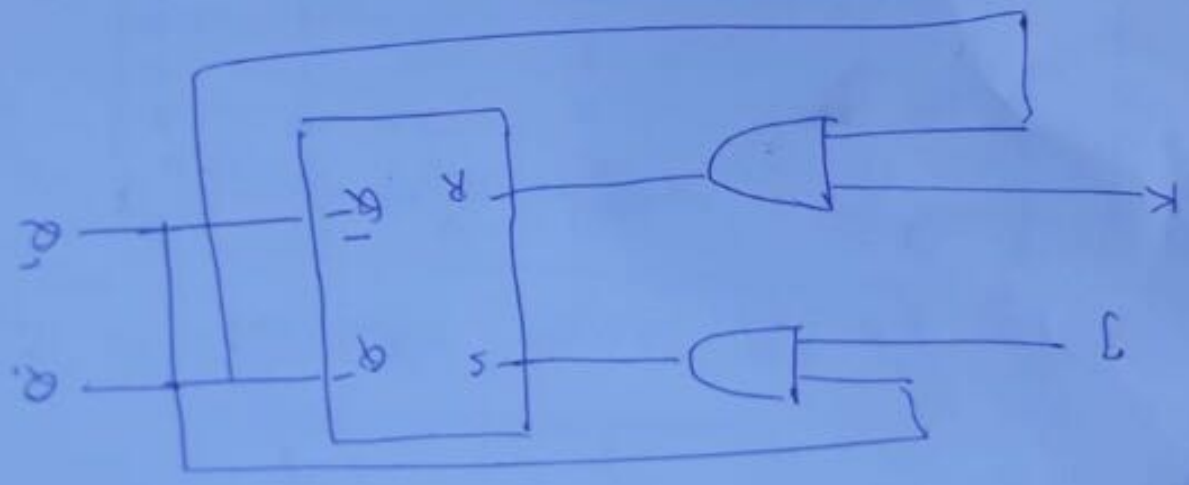
Maurice Sten 5-12 FF

$\overline{S} = \overline{a_n} \overline{J}$

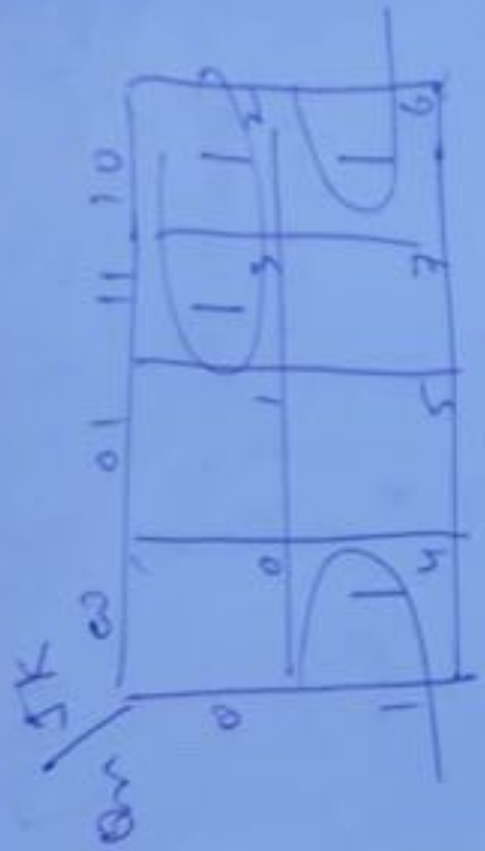
	0	1
a_n	d	d
J	0	1
	10	11

$R = a_n K$

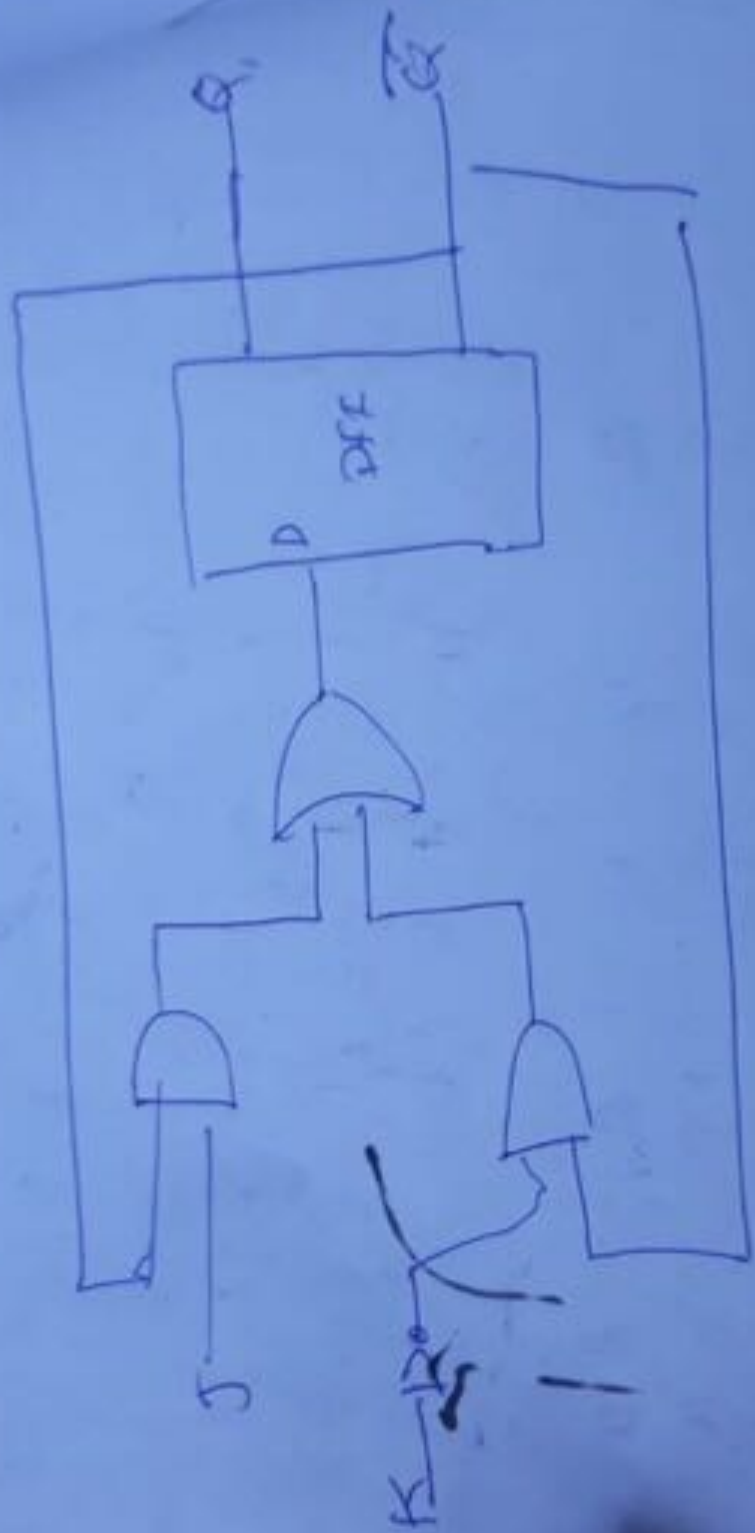
	0	1
a_n	d	1
K	0	1
	10	11



1. Maple
 Realization
 1. 2. 3. 4. 5. 6.



$$D = J\bar{Q}_n + KQ_n$$



Realization of T. FF using S-R FF

1. Available SR
2. Required T

3. Characteristic table for required FF-(T)

Q_m	T	Q_{m+1}
0	0	0
0	1	1
1	0	1
1	1	0

4. Excitation Table for available FF

Q_m	T	Q_{m+1}	S	R
0	0	0	0	d
0	1	1	1	0
1	0	1	d	0
1	1	0	0	1

5. ~~BT~~ Character Equations / Boolean eqns

Q_m	1	3
0	0	1
1	0	1

Q_m	0	1
0	d	0
1	1	0

$$S = \bar{Q}_m T$$

$$R = T \bar{Q}_m$$

Prob

Realization of J-K FF using D FF

Available D
Required J-K

2

Q _n	J	K	Q _{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

3-

Q _n	J	K	Q _{n+1}	D
0	0	0	0	0
0	0	1	0	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

2

Conversion of One FF using other FF

It is possible to implement a flip flop circuit using any other FF

The are 5 step for FF conversion -

1. Identify available & required FF
2. Make characteristic table for required FF
3. Make excitation table for available FF
4. write boolean expression for available FF
5. Draw logic circuit

Ex: Realization of D FF using S-R FF

- 1- Available S-R FF
- Required ~~SR~~ D FF

2. Characteristic Table required (D) ff

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

8. Make excitation table of S-R FF

Q	Q _n		Q _{n+1}		S	R
	Q _n	Q _{n+1}	Q _n	Q _{n+1}		
0	0	0	0	0	0	0
0	0	1	0	1	0	0
1	0	0	1	0	1	0
1	1	1	1	1	0	0

4. Write the boolean expressions for available

Q _n	D	Q _{n+1}	S	R
0	0	0	0	0
0	1	1	1	0
1	0	0	0	1
1	1	1	0	0

Q_n \ D

0	0	1
1	0	1
2	1	1
3	1	1

$$S = D$$

$$R = \bar{D}$$

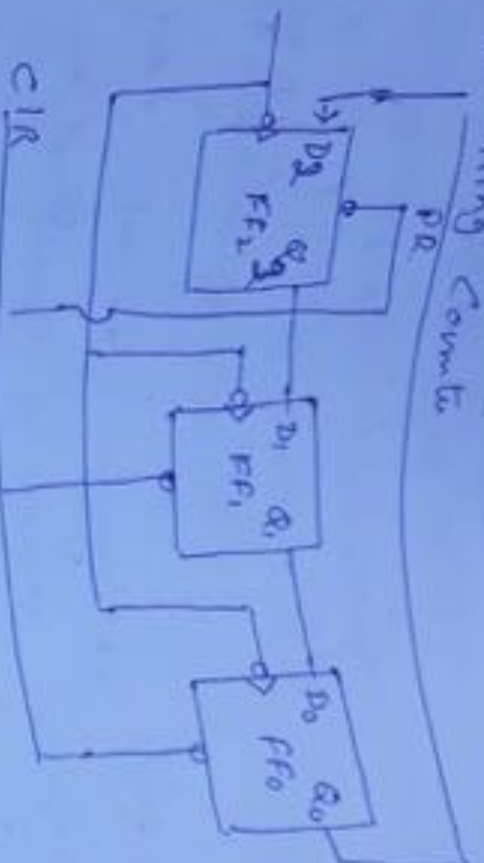
Q_n \ D

0	0	1
1	0	1
2	1	1
3	1	1

3. Draw 1.10 ckt



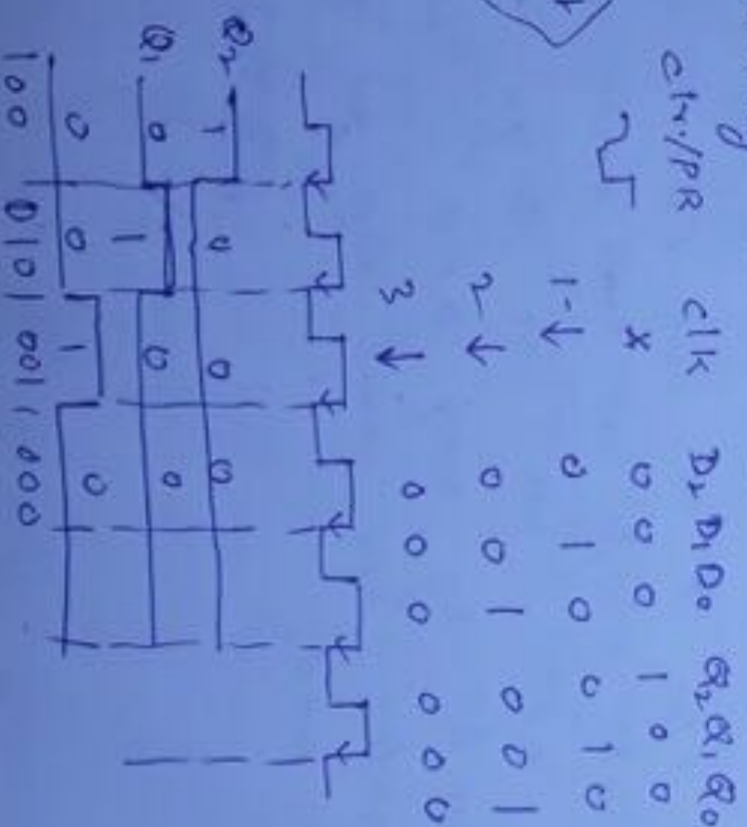
3 Bit Ring Counter



When $PR = 0$ $Q = 1$
 $CLK = 0$ $Q = 0$

Initially all FF are reset to 0 by using $chr. signal$. Before applying clock pulse on FF, we apply PRESET pulse to the first FF so that among the values '1' to the ring counter. For each clock pulse, the state circulates among all the FF again.

only 3 states
 out of 8 states



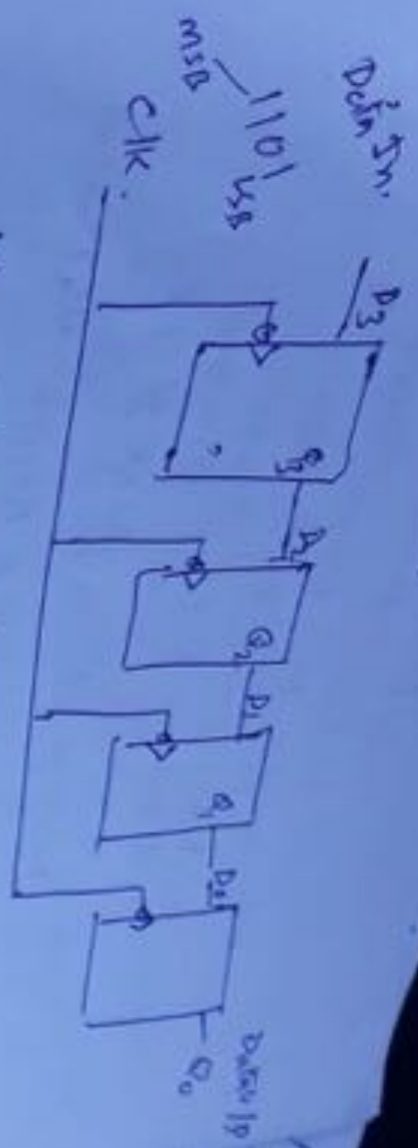
24

a |

1-
2-
3



4 bits SISO



let Data In = 1101

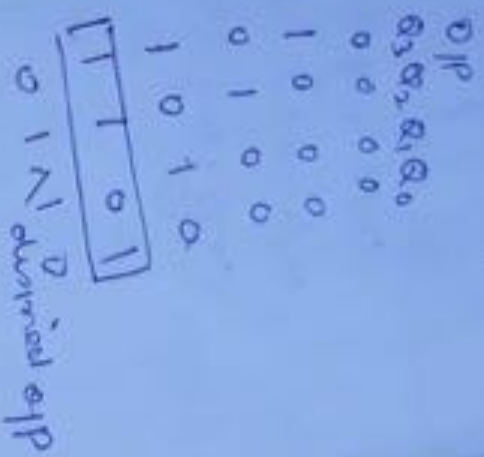
	clk	D ₃	D ₂	D ₁	D ₀	Q ₃ Q ₂ Q ₁ Q ₀	Q ₃ Q ₂ Q ₁ Q ₀
(1)	↓	0	0	0	0	0000	
(2)	↓	1	0	0	0	1000	
(3)	↓	0	1	0	0	0100	
(4)	↓	0	0	1	0	0010	
(5)	↓	0	0	0	1	0001	
(6)	↓	0	0	0	0	0000	
(7)	↓	0	0	0	0	0000	
(8)	↓	0	0	0	0	0000	

4 FF, 4 bits

msb, mff

0000

0001



1 FF
254 FF
4 IF

Registers Application
Binary registers are separated into two parts: registers and combinational logic.

Shift registers are separated into two parts: registers and combinational logic.

(1) Shift registers. which is classified as:

- (i) Ring Counter.
- (ii) Johnson Counter/Decoding/Twisted Ring Counter.

Ring Counter is a sequential logic circuit that is constructed using shift registers.

The main feature of ring counter is that it has a feedback connection from the output of the last flip-flop to the input of the first flip-flop.

The output of the last flip-flop is connected to the input of the first flip-flop. The output of the first flip-flop is connected to the input of the second flip-flop. The output of the second flip-flop is connected to the input of the third flip-flop. The output of the third flip-flop is connected to the input of the fourth flip-flop. The output of the fourth flip-flop is connected to the input of the first flip-flop.

The output of the last flip-flop is connected to the input of the first flip-flop. The output of the first flip-flop is connected to the input of the second flip-flop. The output of the second flip-flop is connected to the input of the third flip-flop. The output of the third flip-flop is connected to the input of the fourth flip-flop. The output of the fourth flip-flop is connected to the input of the first flip-flop.

The output of the last flip-flop is connected to the input of the first flip-flop. The output of the first flip-flop is connected to the input of the second flip-flop. The output of the second flip-flop is connected to the input of the third flip-flop. The output of the third flip-flop is connected to the input of the fourth flip-flop. The output of the fourth flip-flop is connected to the input of the first flip-flop.

Data format 1.9. Classification of Register

Two ways. mainly & primarily, based on the register can be

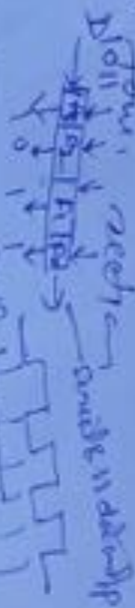
Classified in four: according to output

1. SISO

2. SIPO

3. PISO

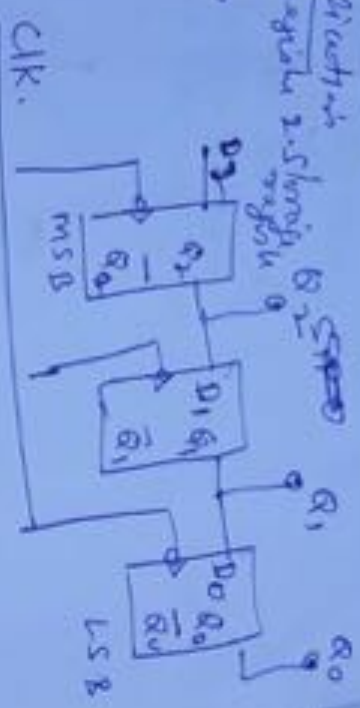
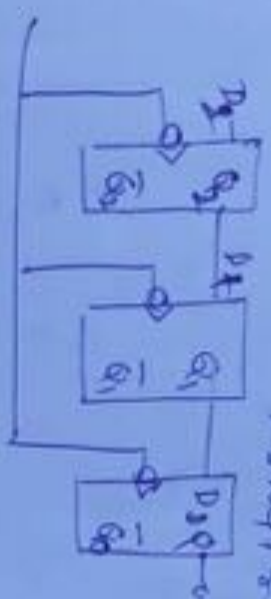
4. PIPO



1.8.1.3.0

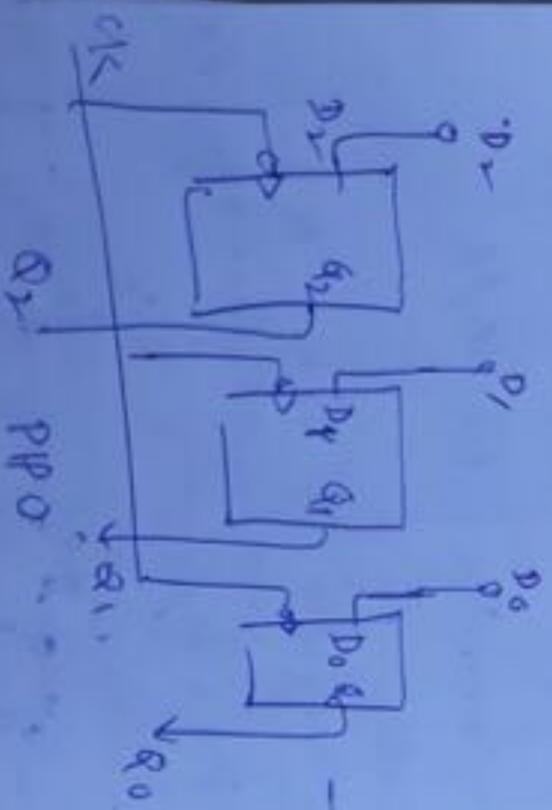
Depending on applications

1. Shift register 2. Shift register

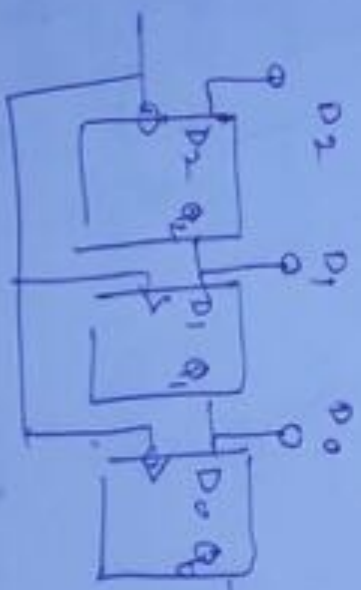


① SISO

② SIPO

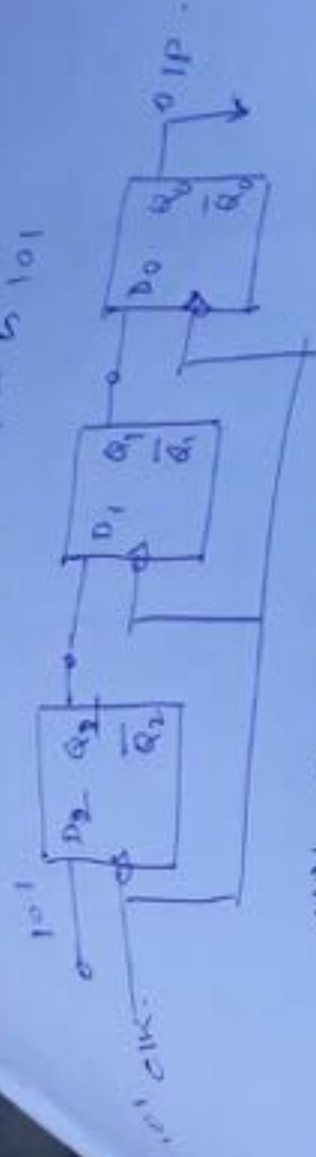


PISO



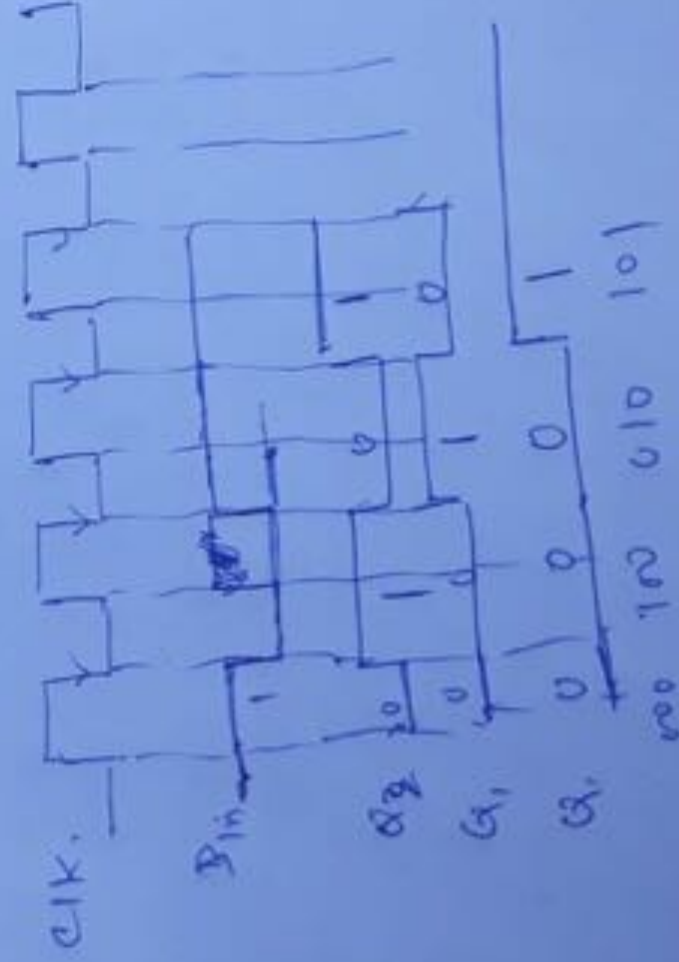
3 Bit SISO Register

with information is 101



CLK	D ₂	D ₁	D ₀	Q ₂ , Q ₁ , Q ₀	Delay in 1st to transfer information
0	0	0	0	0 0 0	
1 ↓	1	0	0	1 0 0	
2 ↓	0	1	0	0 1 0	
3 ↓	1	0	1	1 0 1	

4 ↓	0	1	0	0 1 0	
5 ↓	0	0	1	0 0 1	
6 ↓	0	0	0	0 0 0	



Register

As we know FF are 160 or 144
necessary storage device.
If we want to store 2 bits

we need 2 FF.

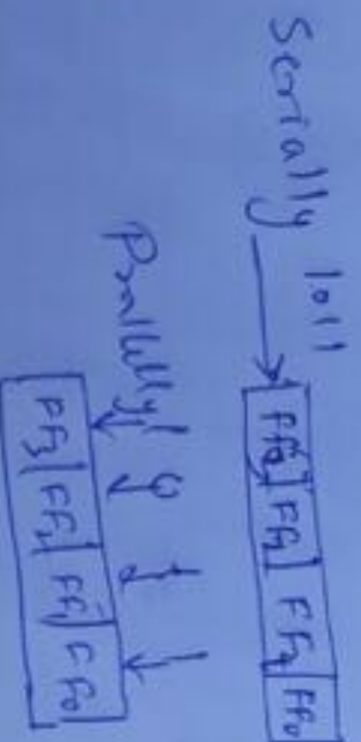
for 3 bits, we need 3 FF.

for n bits, we need n FF

for storing group of bits we need group of FF.
Those group of FF are called registers.
Registers are made up of 1 D FF

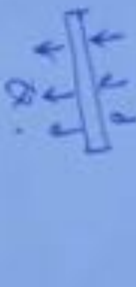
Formats & classification of Registers :- The

formats for 16 way data is entered
in FF is called format. There are
two types of formats entered in FF
serially & Parallelly.



Classification of Registers

1. Depending on T/P & O/P (nature of operation)

- (i) SISO $\rightarrow$ 
- (ii) SIPO $\rightarrow$ 
- (iii) PISO $\rightarrow$ 
- (iv) PIPPO $\rightarrow$ 

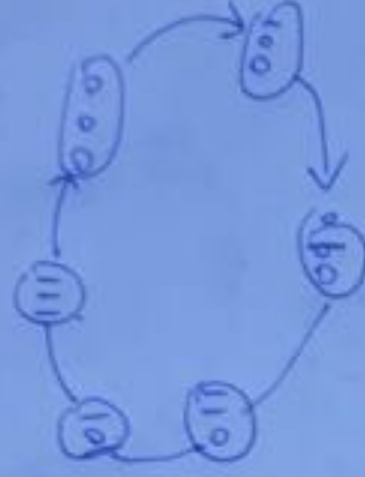
2. Depending on application

- (i) Shift register - shift data left/right
- (ii) Storage register (Buffer register)

0-1-2-3-4-5-7
 $\text{MOD } N_0 = 8$
 $\text{Max Count} = 8-1 \Rightarrow 7$

Excitation Table

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0



P.S

Q_2, Q_1, Q_0

000	001	010	011	100	101	110	111
-----	-----	-----	-----	-----	-----	-----	-----

N.S

Q_2, Q_1, Q_0

000	001	010	011	100	101	110	111
-----	-----	-----	-----	-----	-----	-----	-----

Q_2, Q_1

00	01	11	10
0	0	1	1
1	1	0	0

$$T_2 = Q_0 \bar{Q}_2 Q_1 + Q_0 Q_2 \bar{Q}_1$$

Q_2, Q_1

00	01	11	10
0	0	1	1
1	1	0	0

$$T_1 = Q_0 + Q_2$$

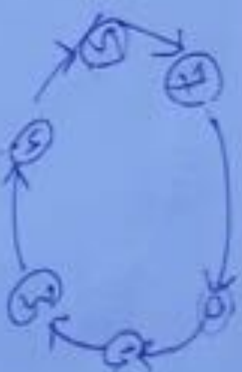
Q_0, Q_2

00	01	11	10
0	0	1	1
1	1	0	0

$$T_0 = Q_0 \bar{Q}_2 + \bar{Q}_0 Q_2$$

Prob

Design asynchronous counter for sequence 0-1-2-4-5-7-6
state diagram.



$Q_2 Q_1 Q_0$	N.S.	T_2	T_1	T_0
000	001	000	0	1
001	011	100	1	0
011	100	21	1	1
100	101	30	0	1
101	111	40	1	0
111	000	51	1	1

$Q_2 Q_1$	T_2	T_1
00	0	1
01	1	0
10	0	1
11	1	0

$Q_2 Q_1$	T_2	T_1
00	0	1
01	1	0
10	0	1
11	1	0

$$T_2 = \bar{Q}_2 Q_1 Q_0 + Q_2 Q_1 \bar{Q}_0$$

$$T_1 = \bar{Q}_0 Q_1 + Q_0 Q_2$$

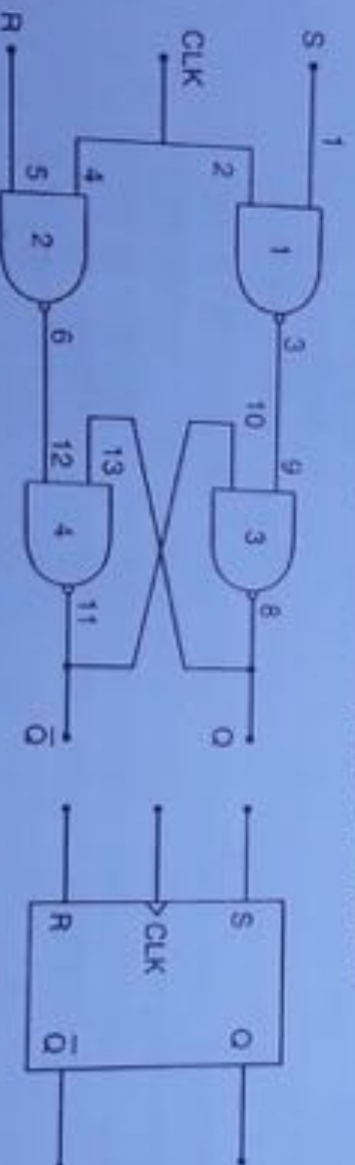
In this circuit, when the clock input is LOW, the output of both the AND gates are LOW and the changes in S and R inputs will not affect the output (Q) of the flip-flop. When the clock input becomes HIGH, the value at S and R inputs will be passed to the output of the AND gates and the output (Q) of the flip-flop will change according to the changes in S and R inputs as long as the clock input is HIGH. In this manner, one or a 0 by applying $S=0, R=1$ (i.e. reset) at any time and then hold that bit of information for any desired period of time by applying a LOW at the clock input. This flip-flop is called clocked S - R flip-flop.

The S - R flip-flop which consists of the basic NOR latch and two AND gates is shown in Fig. 7.10.



Fig. 7.10 Clocked NOR-based S - R flip-flop

The S - R flip-flop which consists of the basic NAND latch and two other NAND gates is shown in Fig. 7.11(a). The S and R inputs control the state of the flip-flop in the same manner as described earlier for the basic (unclocked) S - R latch. However, the flip-flop does not respond to these inputs until the rising edge of the clock signal occurs. The clock pulse input acts as an enable signal for the other two inputs. The outputs of NAND gates 1 and 2 stay at the logic 1 level as long as the clock input remains at 0. This 1 level at the inputs of NAND-based basic S - R latch retains the present state, i.e. no change occurs. The characteristic table of the S - R flip-flop is shown in truth table of Table 7.5. This table shows the operation of the flip-flop in tabular form.

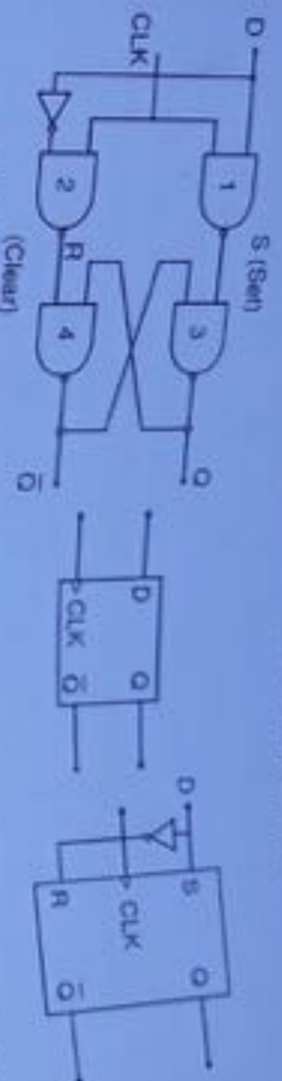


(a) NAND-based S - R flip-flop

(b) Graphic symbol

Fig. 7.11

Given in Table 7.6 from which it is clear that the next state of the flip-flop at time (Q_{n+1}) follows the value of the input D when the clock pulse is applied. As transfer of D -type flip-flop is either used as a delay device or as a latch to store 1 bit of binary information.



(a) Using NAND gates

(b) Logic symbol

(c) Using S-R flip-flop

Fig. 7.13 D flip-flop

Table 7.6 Truth table of D flip-flop

CLK	Input D	Output Q_{n+1}
1	0	0
1	1	1
0	X	No change

7.5.1 State Diagram and Characteristic Equation of D Flip-flop

The state transition diagram for the delay flip-flop is shown in Fig. 7.14.



Fig. 7.14 State diagram of delay flip-flop

From the above state diagram, it is clear that when $D=1$, the next state will be 1; when $D=0$, the next state will be 0, irrespective of its previous state. From the state diagram, one can draw the present state-next state table and the application or excitation table for the Delay flip-flop as shown in Table 7.7 and Table 7.8 respectively.

Table 7.7 Present state-next state table for D flip-flop

Present state Q_n	Delay input D	Next state Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

1. Initially all inputs are 0 and the Q output is 0.
2. When the rising edge of the first clock pulse is 0, inputs are both 0, so the flip-flop is not affected and it remains in the $Q = 0$ state.
3. At the occurrence of the rising edge of the second clock pulse (point *e*), the S and R inputs are both 1. Thus, the flip-flop is not affected and it remains in the $Q = 0$ clock pulse.
4. When the third clock pulse makes its positive transition (point *c*), $S = 0$ and $R = 1$, which causes the flip-flop to reset to the 0 state.
5. The fourth pulse sets the flip-flop once again to the 1 state (point *e*), because $S = 1$ and $R = 0$ when its positive edge occurs.
6. The fifth pulse finds that $S = 1$ and $R = 0$ when its positive edge occurs. However, Q is already high, so it remains in that state.
7. The $R = 1$ and $S = 1$ condition should not be used as it results in an indeterminate condition.



Fig. 7.12 Waveforms of S-R flip-flop

7.5 D FLIP-FLOP

The D (delay) flip-flop has only one input called the Delay (D) input and two outputs Q and $\bar{Q}$. It can be constructed from an S-R flip-flop by inserting an inverter between S and R and assigning the symbol D to the S input. The structure of D flip-flop is shown in Fig. 7.13(a). Basically, it consists of a NAND flip-flop with a gating arrangement on its inputs. It operates as follows:

1. When the CLK input is LOW, the D input has no effect, since the set and reset inputs of the NAND flip-flop are kept HIGH.
2. When the CLK goes HIGH, the Q output will take on the value of the D input. If $CLK = 1$ and $D = 1$, the NAND gate-1 output goes 0 which is the $\bar{S}$ input of the basic NAND-based S-R flip-flop and NAND gate-2 output goes 1 which is the $\bar{R}$ input of the basic NAND-based S-R flip-flop. Therefore, for $\bar{S} = 0$ and $\bar{R} = 1$, the flip-flop output will be 1, i.e. it follows D input. Similarly, for $CLK = 1$ and $D = 0$, the flip-flop output will be 0. If D changes while the CLK is HIGH, Q will follow and change quickly.

The logic symbol for the D flip-flop is shown in Fig. 7.13(b). A simple way of building a delay D flip-flop is shown in Fig. 7.13(c). The truth table of D flip-flop is

- Initially all inputs are 0 and the output Q is 1.
- When the first clock pulse goes positive (point a), the $J = 0$, $K = 1$ condition exists. Thus, the flip-flop will be cleared to the $Q = 0$ state.
- The second clock pulse finds $J = K = 1$ when it makes its positive transition (point c). This causes the flip-flop to toggle to its opposite state, $Q = 1$.
- At point e on the clock waveform, J and K are both 0, and hence the flip-flop does not change state on this transition.
- At point g , $J = 1$ and $K = 0$. This is the condition that sets Q to the 1 state. However, it is already 1, and it will remain in the same state.
- At point i , $J = 1 = K$. The flip-flop toggles to its opposite state. The same toggling of state occurs at point k since $J = K = 1$.



Fig. 7.17 Waveforms of the J-K flip-flop operation

7.6.1 State Diagram and Characteristic Equation of J-K Flip-flop

The state transition diagram for J-K flip-flop can be drawn as shown in Fig. 7.18.



Fig. 7.18 State diagram of J-K flip-flop

From the above state diagram, one can easily understand that the state transition from 0 to 1 takes place whenever J is asserted (i.e. $J = 1$) irrespective of K value. Similarly, state transition from 1 to 0 takes place whenever K is asserted (i.e. $K = 1$) irrespective of the value of J . Also, the state transition from 0 to 0 occurs whenever $J = 0$, irrespective of the value of K , and the state transition from 1 to 1 occurs whenever $K = 0$, irrespective of J value.

From the above state diagram and truth table (Table 7.9) of J-K flip-flop, the present state-next state table and application table or excitation table for J-K flip-flop are shown in Table 7.10 and Table 7.11 respectively.

Table 7.9 Truth table of $J-K$ flip-flop

CLK	Inputs		Output Q_{n+1}	Action
	J	K		
X	0	0	Q_n	No change
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	$\bar{Q}_n$	Toggle

Case 1 When $J=0$, whatever be the value of $\bar{Q}_n$ (0 or 1), the output of the first AND gate is 0. Similarly, when $K=0$, whatever be the value of Q_n (0 or 1), the output of the second AND gate is also 0. Therefore, when $J=0$ and $K=0$, the inputs to the basic flip-flop are $S=0$ and $R=0$. This condition forces the flip-flop into the same state.

Case 2 When $J=0$, $K=1$ and the previous state of flip-flop is SET (i.e. $Q_n=1$, $\bar{Q}_n=0$), then $S=J\bar{Q}_n=0 \times 0=0$ and $R=KQ_n=1 \times 1=1$. Since $S=0$ and $R=1$, the flip-flop RESETS on the application of a clock pulse. But, if the flip-flop is already RESET (i.e., $Q_n=0$ and $\bar{Q}_n=1$), then the application of $J=0$ and $K=1$ will not alter the state of the flip-flop ($S=J\bar{Q}_n=0 \times 1=0$ and $R=KQ_n=1 \times 0=0$), and it remains in the RESET state.

Case 3 When $J=1$, $K=0$ and the previous state of the flip-flop is RESET ($Q_n=0$ and $\bar{Q}_n=1$), then $S=J\bar{Q}_n=1 \times 1=1$ and $R=KQ_n=0 \times 0=0$. Since $S=1$ and $R=0$, the flip-flop will SET on the application of a clock pulse. But, if the flip-flop is already set (i.e. $Q_n=1$, $\bar{Q}_n=0$), then the inputs $J=1$, $K=0$ will make $S=0$ and $R=0$, and the flip-flop remains in the SET state.

Case 4 The condition $J=1$, $K=1$ deserves special attention. When $J=1$, $K=1$ and the previous state is a SET state (i.e. $Q_n=1$, $\bar{Q}_n=0$), then $S=J\bar{Q}_n=1 \times 0=0$ and $R=KQ_n=1 \times 1=1$. Since $S=0$ and $R=1$, the flip-flop RESETS on the application of a clock pulse, i.e. the flip-flop toggles from SET to RESET state. Again for $J=1$, $K=1$, if the previous state is a RESET state (i.e. $Q_n=0$, $\bar{Q}_n=1$), then $S=J\bar{Q}_n=1 \times 1=1$ and $R=KQ_n=1 \times 0=0$. Since $S=1$ and $R=0$, the flip-flop on the application of a clock pulse will toggle to the SET state. Hence, when $J=1$ and $K=1$, the flip-flop toggles on the application of the next clock pulse. Toggle means to switch to the opposite state. The flip-flop will continuously change its state when $J=K=1$ and CLK=HIGH, resulting in an unstable output.

It is very important to realise that because of the feedback connection in the $J-K$ flip-flop, a CLK pulse that remains in the 1 state while both J and K are equal to 1 will cause the output to complement again and again until the clock pulse goes back to 0. To avoid this undesirable operation, the clock pulse must have a time duration that is shorter than the propagation delay time of the flip-flop. This is a restrictive requirement, since the operation of the circuit depends on the width of the pulse. The restriction on the pulse width can be eliminated with a master slave or edge triggered construction which will be discussed shortly.

The operation of the $J-K$ flip-flop is illustrated by the waveforms in Fig. 7.17, which can be analysed as follows:

Table 7.8 Application or excitation table for D flip-flop

Q_n	Q_{n+1}	Excitation input D
0	0	0
0	1	1
1	0	0
1	1	1

Using the Present state–Next state table, the K-map for the next state (Q_{n+1}) of the Delay flip-flop can be drawn as shown in Fig. 7.15, and the simplified expression for Q_{n+1} can be found as follows.

$Q_n \backslash D$	0	1
0	0	1
1	0	1

Fig. 7.15 Next state (Q_{n+1}) map for D flip-flop

From the above K-map, the characteristic equation for Delay flip-flop is

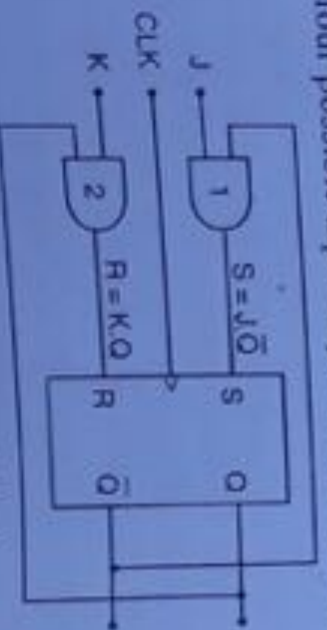
$$Q_{n+1} = D$$

Hence, in a Delay flip-flop, the next state follows the Delay input as shown by the characteristic equation.

7.6 J-K FLIP-FLOP

A J-K flip-flop has a characteristic similar to that of an S-R flip-flop. In addition, the indeterminate condition of the S-R flip-flop is permitted in it. Inputs J and K behave like inputs S and R to set and reset the flip-flop, respectively. When $J = K = 1$, the flip-flop output toggles, i.e. switches to its complement state; if $\bar{Q} = 0$, it switches to $\bar{Q} = 1$ and vice versa.

A J-K flip-flop can be obtained from the clocked S-R flip-flop by augmenting two AND gates as shown in Fig. 7.16(a). The data input J and the output $\bar{Q}$ are applied to the first AND gate, and its output ($J\bar{Q}$) is applied to the S input of S-R flip-flop. Similarly, the data input K and the output $\bar{Q}$ are connected to the second AND gate and its output ($K\bar{Q}$) is applied to R input of S-R flip-flop. The graphic symbol of J-K flip-flop is shown in Fig. 7.16(b) and the truth table is shown in Table 7.9. The output for the four possible input sequences are described below.



(a) J-K flip-flop using S-R flip-flop



(b) Graphic symbol of J-K flip flop

Fig. 7.16

diagram, we can draw the present state, next state table and apply them to a Karnaugh map for the trigger flip-flop as shown in Table 7.13 and Table 7.14 respectively.

Table 7.13 Present state, next state table for T flip-flop

Q	T	Q _{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Table 7.14 Application of excitation table for T flip-flop

Q	Q _{n+1}	Excitation input T
0	0	0
0	1	1
1	0	1
1	1	0

From the Table 7.13, the K-map for the next state (Q_{n+1}) of trigger flip-flop can be drawn as shown in Fig. 7.22, and the simplified expression for Q_{n+1} can be found as follows.



Fig. 7.22 Next state (Q_{n+1}) map for T flip-flop

From the K-map shown in Fig. 7.22, the characteristic equation for trigger flip-flop is

$$Q_{n+1} = T\bar{Q}_n + \bar{T}Q_n$$

So, in a trigger flip-flop, the next state will be the complement of the previous state when $T = 1$.

7.8 TRIGGERING OF FLIP-FLOPS

Flip-flops are synchronous bistable devices. The term *synchronous* means that the changes in the output occur at a specified point on a triggering input called the clock; that is, changes in the output occur in synchronisation (in time) with the clock. Based on the specific interval or point in the clock during or at which triggering of flip-flop takes place, it can be classified into two different types.

- Level triggering
- Edge triggering.



(a) Block diagram of T flip-flop



(b) T flip-flop using a J-K flip-flop

Fig. 7.20

When the T input is in the 0 state (i.e. $J = K = 0$) prior to a clock pulse, the Q output will not change with clocking. When the T input is at a 1 (i.e. $J = K = 1$) level prior to clocking, the output will be in the $\bar{Q}$ state after clocking. In other words, if the T input is a logical 1 and the device is clocked, the output will change state regardless of what output was prior to clocking. This is called *toggling* hence the name T flip-flop.

Table 7.12 Truth table of T flip-flop

Q_n	T	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

The truth table shows that when $T = 0$, then $Q_{n+1} = Q_n$, i.e., the next state is the same as the present state and no change occurs. When $T = 1$, then $Q_{n+1} = \bar{Q}_n$, i.e., the state of the flip-flop is complemented.

Application of T flip-flop T-type flip-flop is most often seen in counters and sequential counting networks because of its inherent divide-by-2 capability. When a clock pulse is applied, the output changes state once every input cycle, thus repeating one cycle for every two input cycles. This is the action required in many cases for binary counters.

7.7.1 State Diagram and Characteristic Equation of T Flip-flop

The state transition diagram for the Trigger flip-flop is shown in Fig. 7.21.



Fig. 7.21 State diagram of trigger flip-flop

From the above state diagram, it is clear that when $T = 1$, the flip-flop changes or toggles its state irrespective of its previous state; when $T = 1$ and $Q_n = 0$, the next state will be 1; when $T = 1$ and $Q_n = 1$, the next state will be 0. Similarly, one can understand that when $T = 0$, the flip-flop retains its previous state. From the above state

Table 7.18 Present state–next state table for $J-K$ flip-flop

Present state Q_n	Inputs		Next state Q_{n+1}
	J	K	
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Table 7.11 Application or excitation table for $J-K$ flip-flop

Q_n	Q_{n+1}	Excitation inputs	
		J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

From the Table 7.10, a Karnaugh map for the next state transition (Q_{n+1}) can be drawn as shown in Fig. 7.19, and the simplified logic expression which represents the characteristic equation of $J-K$ flip-flop can be found as follows.

From the K -map shown in Fig. 7.19, the characteristic equation of $J-K$ flip-flop can be written as

$$Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$$

Q_n	JK			
	00	01	11	10
0	0	0	1	1
1	1	0	0	1

Fig. 7.19 Next state (Q_{n+1}) map for $J-K$ flip-flop

7.7 T FLIP-FLOP

Another basic flip-flop, called the T or *trigger* or *toggle* flip-flop, has only a single data (T) input, a clock input and two outputs Q and $\bar{Q}$. The T -type flip-flop is obtained from a $J-K$ flip-flop by connecting its J and K inputs together. The designation T comes from the ability of the flip-flop to “toggle” or complement its state.

The block diagram of a T flip-flop and its circuit implementation using a $J-K$ flip-flop are shown in Fig. 7.20. The J and K inputs are wired together. The truth table for T flip-flop is shown in Table 7.12.

In addition to the above synchronous inputs, most clocked flip-flops have one or more *asynchronous inputs* whose effect on the flip-flop output is independent of the synchronous input and the clock input. Asynchronous inputs can be used to set the flip-flop to the 1 state, or clear the flip-flop to the 0 state at any time, regardless of the condition at the other inputs. The asynchronous inputs are *override inputs*, since they can be used to override all other inputs in order to place the flip-flop in one state or the other.

Figure 7.27 shows a clocked J - K flip-flop with $\overline{\text{PRESET}}$ ($\overline{\text{PR}}$) and $\overline{\text{CLEAR}}$ ($\overline{\text{CLR}}$), or *direct set* and *direct reset* inputs. These are active LOW inputs. An active LOW level at the preset input will SET the flip-flop, and an active LOW level on the clear input will RESET it. These asynchronous inputs are activated by a 0 level, as indicated by the small bubbles on the flip-flop symbol. Both the preset and clear inputs must be kept HIGH for synchronous operation. Truth Table 7.17 shows how these inputs operate. Simultaneous LOW levels on preset and clear are forbidden since it leads to an ambiguous condition.

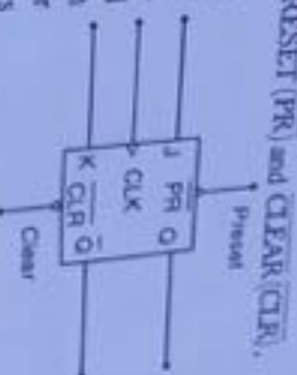


Fig. 7.27 Logic symbol for a J - K flip-flop with active-LOW preset and clear inputs

Table 7.17 Truth table

Inputs		Outputs		Action
Preset	Clear	Q_{n+1}	$\overline{Q}_{n+1}$	
0	0	?	?	Forbidden
0	1	1	0	Set
1	0	0	1	Reset
1	1	Q_n	$\overline{Q}_n$	No change

The logic diagram for an edge-triggered J - K flip-flop with preset $\overline{\text{PR}}$ and clear $\overline{\text{CLR}}$ inputs is shown in Fig. 7.28. The asynchronous inputs are directly connected to the latch portion of the flip-flop so that they override the effect of the synchronous inputs, J , K and the clock.

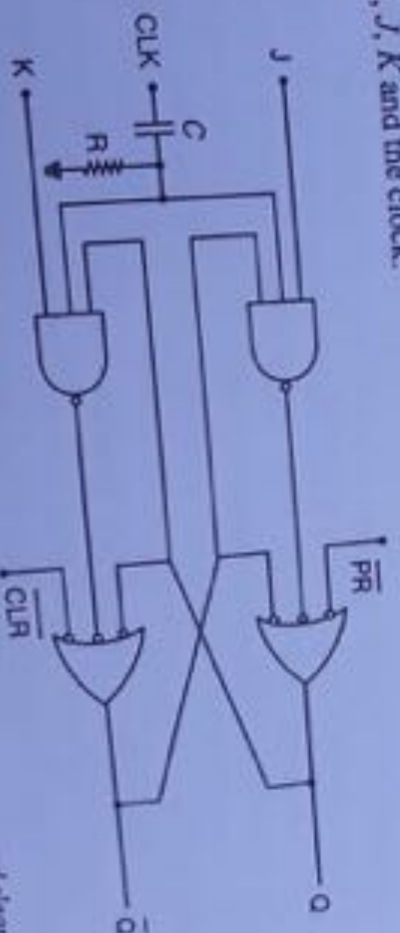


Fig. 7.28 Logic diagram for a basic J - K flip-flop with active-LOW preset and clear inputs
From Fig. 7.28, it is clear that if a constant 0 is applied to the $\overline{\text{PR}}$ (preset) input, the flip-flop output (Q) will become 1 regardless of the status of the other inputs.

A clock pulse starts from an initial value of 0, goes momentarily to 1, and after a time, returns to its initial 0 value. The time interval from the application of the time, the output transition occurs is a critical factor that needs further investigation.

Self-triggering in flip-flops

7.8-1 Level triggered S-R flip-flop In the previous section, it was shown that the while discussing clocked S-R flip-flop, i.e., enables the flip-flop, when the clock pulse goes high. While discussing the flip-flop, it is said to be level triggered flip-flop. Since the flip-flop changes its state when the clock is positive, it is termed as Positive level triggered flip-flop. If the clock input triggers the flip-flop is said to be level triggered flip-flop. If the clock input and the output are both high, it is called Negative level triggered flip-flop.

The main drawback of level triggering is that, as long as the clock is positive or negative, the flip-flop changes its state more than once or many times for the change in inputs. If the inputs are made stable for the entire clock duration, then the output changes only once. On the other hand, if the frequency of input change is higher than the input clock frequency, the output of the flip-flop undergoes multiple changes when the clock is positive or negative. When the clock becomes unasserted (i.e. clock = 0), the clock is positive or negative. This can be overcome in the output of the flip-flop reflects the last change in its inputs. This can be overcome in Master-Slave flip-flops and Edge-triggered flip-flops where the flip-flops change state only once for a clock.

7.8.2 Edge Triggering in Flip-flops

7.8.2 Edge-triggering

A clock pulse goes through two signal transitions from 0 to 1 and returns from 1 to 0. As shown in Fig. 7.23, a positive transition is defined as the *positive edge* and a negative transition as the *negative edge*. This definition applies also to negative pulses. The term *edge-triggered* means that the flip-flop changes its state either at the positive edge (rising or leading edge) or at the negative edge (falling or trailing edge) of the clock pulse and is sensitive to its inputs only at this transition of the clock.

Negative pulse

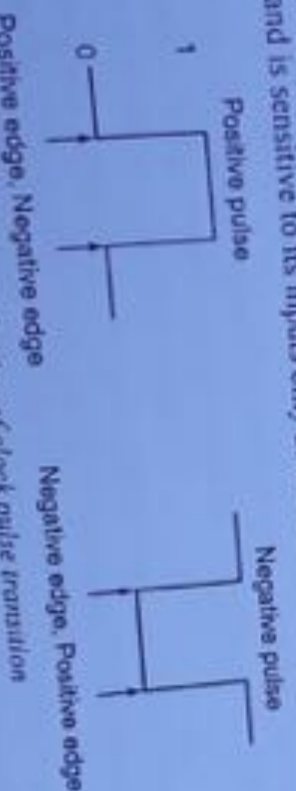


Fig. 7.23 Definition of clock phase

One way to make the flip-flop insensitive to the input signal is to use an RC (resistor-capacitor) circuit as shown in Fig. 15-15. An RC circuit is a circuit in which the capacitor can charge fully through the resistor. By deliberate design, the RC time constant is much smaller than the clock's pulse width. Because of this, the capacitor can charge fully through the resistor. When the clock goes high, this exponential charging produces a narrow positive voltage spike across the resistor. Later, the trailing edge of the pulse results in a narrow negative spike across the resistor. The circuit that generates a spike in response to a momentary change of input signal is called an *edge trigger*. Edge triggering is achieved by designing the flip-flop to neglect one spike and trigger on the occurrence of the other spike.

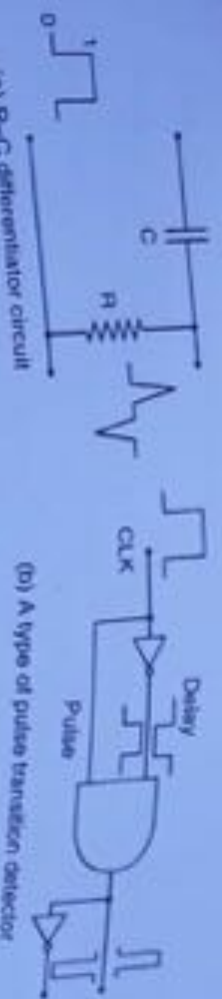


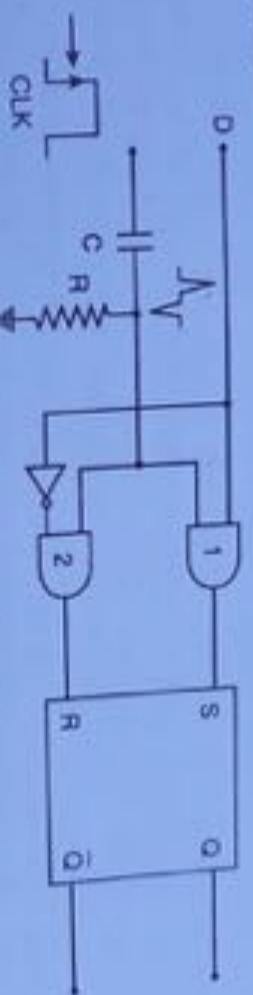
Fig. 7.24

Another type of a pulse transition detector is shown in Fig. 7.24(b). There is a small delay on one input of the AND gate due to propagation delay of the NOT gate. So, the inverted clock pulse arrives at the gate input a few nanoseconds after the true clock pulse. This produces an output spike with a time duration of only a few nanoseconds.

In certain applications, triggering on the negative edge is better suited. In this case, an inverter complements the clock pulse.

7.8.3 Edge-triggered D Flip-flop

The circuit of an edge-triggered D flip-flop is shown in Fig. 7.25. Here, a differentiating circuit comprising of a capacitor and a resistor is inserted between the clock and the input to the AND gates. The time constant (RC) is made much smaller than the clock's pulse width.

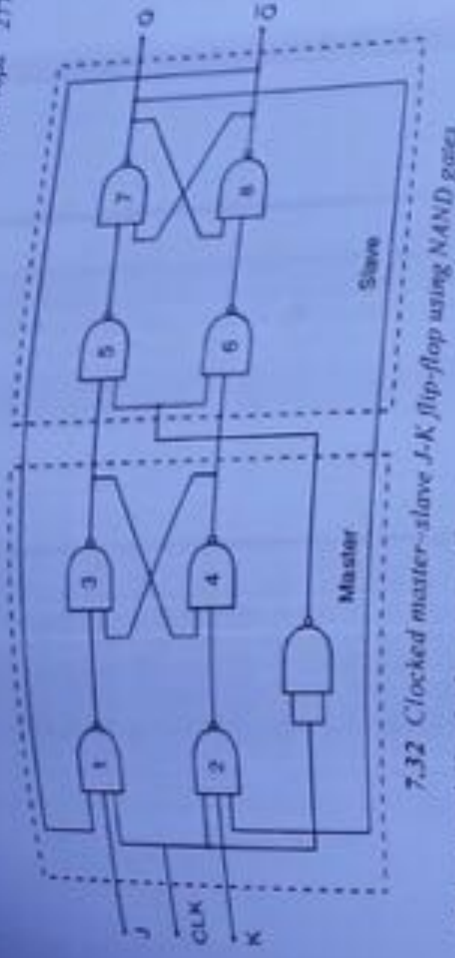
Fig. 7.25 Edge-triggered D flip-flop

The narrow positive spike enables both the AND gates for an instant. During the time of narrow positive spike, D and its complement hit the inputs of the flip-flop, forcing Q to set or reset (unless Q already equals D).

The truth table given in Table 7.15 shows the action of a positive edge-triggered D flip-flop. When the CLK input is LOW, the flip-flop output retains its previous state, irrespective of whether $D = 0$ or 1. When the leading edge of the clock pulse, designated by the upward arrow, is passed through the AND gates, the flip-flop output (Q) takes on the value of D . During the trailing edge of the CLK, designated by a downward arrow, Q remains in its last state, irrespective of the value of D .

Table 7.15 Truth table of the positive edge-triggered D flip-flop

CLK	D	Q_{n+1}	Action
0	X	Q_n	Last state
↑	0	0	Reset
↑	1	1	Set
↓	X	Q_n	Last state



If $J = 1$ and $K = 0$, the master flip-flop sets on the positive clock edge. The HIGH $Q(1)$ output of the master drives the input (J) of the slave. So, when the negative clock edge hits, the slave also sets. The slave flip-flop copies the action of the master flip-flop.

If $J = 0$ and $K = 1$, the master resets on the leading edge of the CLK pulse. The HIGH $\bar{Q}$ output of the master drives the input (K) of the slave flip-flop. Then, the slave flip-flop resets at the arrival of the trailing edge of the CLK pulse. Once again, the slave flip-flop copies the action of the master flip-flop.

If $J = K = 1$, the master flip-flop toggles on the positive clock edge and the slave toggles on the negative clock edge. The condition $J = K = 0$ input does not produce any change.

Master-Slave flip-flops operate from a complete clock pulse, and the outputs change on the negative transition.

7.11 REALISATION OF ONE FLIP-FLOP USING OTHER FLIP-FLOPS

It is possible to implement a flip-flop circuit using any other flip-flop. The general block diagram connection for such realisation is shown in Fig. 7.33.

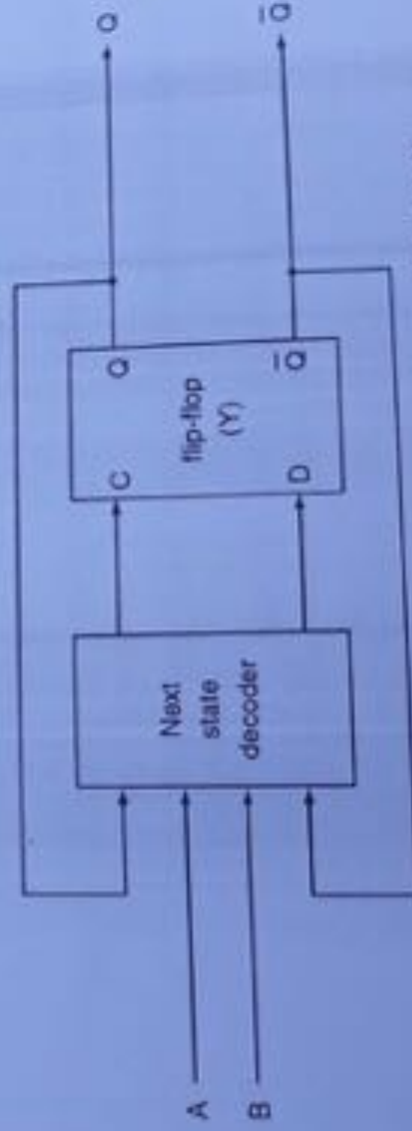


Fig. 7.33 Block diagram to realise flip-flop (X) using flip-flop (Y)

Similarly, a constant LOW (0) at the $\overline{\text{CLK}}$ (clear) input holds the $\overline{Q}$ output of the flip-flop in the HIGH state, i.e. $Q = 0$. Thus, asynchronous inputs can be used to hold the flip-flop in a particular state for any desired interval of time.

Some flip-flops have asynchronous inputs that are activated by 1's (active HIGH) rather than by 0's. The small bubble on the preset and clear input is not shown for these flip-flops.

The illustration of the operation of the asynchronous inputs is shown in Fig. 7.29. The synchronous inputs J and K are held HIGH as CLK pulses are applied. The flip-flop will toggle on each negative-going transition as long as the *preset* and *clear* inputs are both HIGH when the CLK edge occurs. This happens at points *a*, *c*, *d* and *f*. The output Q can also change states in response to a LOW at *preset* or *clear*. The low *preset* causes Q to go to the 1 state immediately, regardless of the CLK state (point *b*). Similarly, the LOW at *clear* causes Q to go to 0 immediately (point *e*). Note that this LOW at *clear* prevents the fourth CLK edge from toggling Q ; this is the override feature.

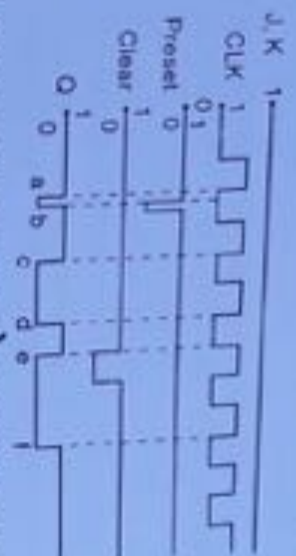


Fig. 7.29 Asynchronous inputs can cause flip-flop to change states immediately without waiting for clock transition

7.10 MASTER-SLAVE FLIP-FLOPS

7.10.1 The Race-around Condition

Before the development of edge-triggered flip-flops, the timing problem in level-triggered flip-flops was often handled by Master-Slave flip-flops.

The condition $S = 1$ and $R = 1$ we have seen earlier is not allowed in an S-R flip-flop. This is eliminated in the J-K flip-flop by using the feedback connection from the $\overline{Q}$ output to the inputs of the gates 1 and 2. Because of the feedback connection $Q(\overline{Q})$ at the input to $K(J)$, the input will change during the clock pulse (CLK = 1), if the output changes the state. Consider, for example, that the inputs are $J = K = 1$ and $\overline{Q} = 0$. When a clock pulse of width t_p , as shown in Fig. 7.30(a) is applied, the output will change from 0 to 1 after a time interval Δt , where Δt is the propagation delay through two NAND gates in series. Now, after time Δt , we have $J = K = 1$ and $\overline{Q} = 1$ and after another interval of Δt , output $\overline{Q}$ will become 0. Hence, the output will oscillate back and forth between 0 and 1 in the duration t_p of the clock pulse width. So, at the end of the clock pulse, the value of $\overline{Q}$ is ambiguous. This situation is known as a *race-around condition*.

7.0.4 Edge-triggered J-K Flip-Flop

The logic diagram of a basic positive edge-triggered J-K Flip-Flop is shown in Fig. 7.26. Note that the $\bar{Q}$ output is connected back to the input of NAND gate-1, and the Q output is connected back to the input of NAND gate-2.

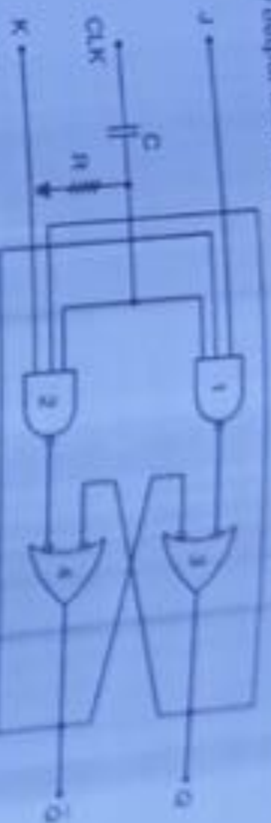


Fig. 7.26 Logic diagram for a positive edge-triggered J-K flip-flop

Let us start by assuming that the flip-flop is RESET (i.e. $\bar{Q} = 0$) and $J = 1$ and $K = 0$. When a CLK pulse occurs, a leading-edge spike is passed through NAND gate-1 because $\bar{Q} = 1$ and $J = 1$. This causes the latch portion of the flip-flop to change to the SET state.

If $J = 0$ and $K = 0$, the flip-flop will retain its present state when the leading edge of the clock pulse occurs. This results in a no change condition.

If $J = 1$ and $K = 1$ and the flip-flop is RESET, then $\bar{Q} = 1$ enables NAND gate-1, so that the clock spike passes through to SET the flip-flop. Now, there is a HIGH on Q which allows the next clock spike to pass through NAND gate-2 and RESET the flip-flop. This is called *toggle operation*.

The operation of the edge-triggered J-K flip-flop is summarised in truth table given in Table 7.16. The truth table for a negative edge-triggered device is the same except that it is triggered on the trailing edge of the CLK pulse.

Table 7.16 Truth table of edge-triggered JK flip-flop

CLK	Inputs		Outputs		Action
	J	K	Q_{n+1}	$\bar{Q}_{n+1}$	
$\uparrow$	0	0	Q_n	$\bar{Q}_n$	No change
$\uparrow$	0	1	0	1	Reset
$\uparrow$	1	0	1	0	Set
$\uparrow$	1	1	$\bar{Q}_n$	Q_n	Toggle

7.9 ASYNCHRONOUS INPUTS IN FLIP-FLOPS

So far, in our discussion, we have encountered S, R, J, K, D and T inputs in the flip-flops. These inputs, normally referred to as *control inputs*, are also called as *synchronous inputs*. For such synchronous inputs, the flip-flop changes its state in synchronisation with the clock input only. So, the asynchronous control inputs must be used along with a clock signal to trigger a change in the flip-flop.

To realise a flip-flop (X) using another flip-flop (Y), the flip-flop (Y) along with a combinational circuit, called NEXT state decoder, is used which functions like flip-flop (X). For this to be realised, for each set of inputs of flip-flop (X), i.e. A & B and cause the flip-flop (Y) to make transition into the flip-flop (Y), i.e. C & D that will procedure for such realisation can be summarised in the following steps:

- (i) Obtain a clear word description of the desired flip-flop (X)
- (ii) Obtain a Present State-Next State (PS-NS) table for the desired flip-flop (X)
- (iii) Using the excitation table or application table of the chosen flip-flop (Y), append the next state code or the excitation input values to the above present state-next state table.
- (iv) Using K -maps, simplify the logic expressions for excitation inputs of flip-flop (Y) and design the next state decoder logic.
- (v) Draw a circuit for the desired flip-flop (X) using next state decoder logic and the chosen flip-flop (Y) as shown in the block diagram.

The above five steps can be clearly explained in the following sections.

7.11.1 Realisation of Delay Flip-flop using S-R Flip-flop

Consider the realisation of Delay flip-flop using S-R flip-flop. Here, the desired flip-flop (X) is Delay flip-flop and the chosen flip-flop (Y) is S-R flip-flop. The basic block diagram is shown in Fig. 7.34.

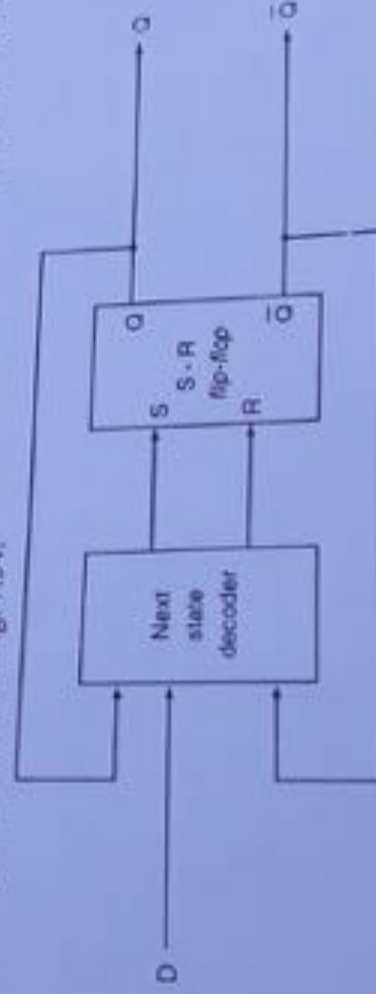


Fig. 7.34 Block diagram of D-flip-flop using S-R flip-flop

Step 1

The operation of Delay flip-flop is well known, and hence there is no need for word description.

Step 2

The Present State-Next State table for D flip-flop can be drawn as shown in Table 7.17.

Table 7.17 PS-NS table for D flip-flop

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

Step 3

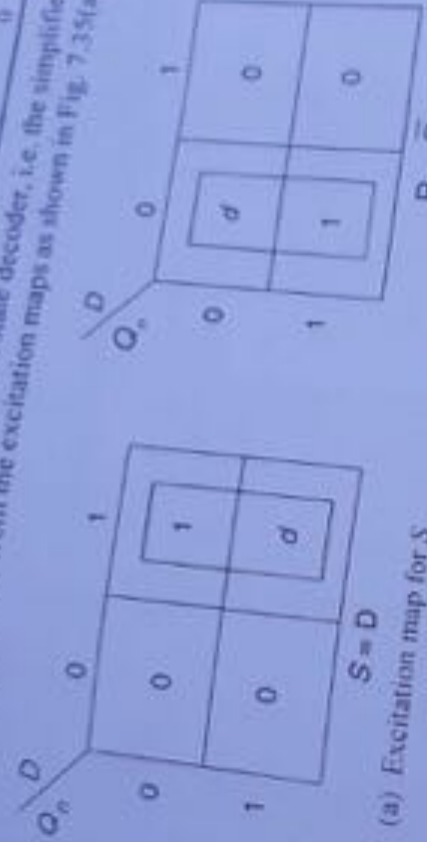
Using the application table of S - R flip-flop given in Table 7.4, the next state codes, i.e. S & R values, can be augmented in the above pn - ns table as shown in Table 7.18.

Table 7.18 Excitation table for D flip-flop realisation using S - R flip-flop

Q_n	D	Q_{n+1}	Excitation inputs	
			S	R
0	0	0	0	d
0	1	1	1	0
1	0	0	0	1
1	1	1	1	0

Step 4

In this step, one can design the next state decoder, i.e. the simplified expressions for S and R , from the excitation maps as shown in Fig. 7.35(a) and (b).



Step 5

From the above step, $S = D$ and $R = \bar{D}$. Now, the circuit for Delay flip-flop using S - R flip-flop can be drawn as shown in Fig. 7.36, with a single NOT gate in the next state decoder logic. Note that the designed circuit confirms with the earlier construction of the delay flip-flop using S - R flip-flop.

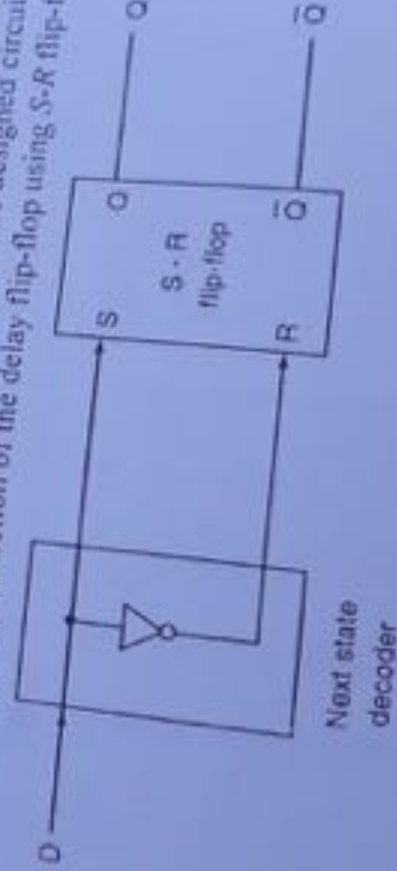


Fig. 7.36 D flip-flop using S - R flip-flop

7.11.2 Realisation of J - K Flip-flop using S - R Flip-flop

In this section, let us consider the realisation of J - K flip-flop using S - R flip-flop. Here, the desired flip-flop (X) is a J - K flip-flop and the chosen flip-flop (Y) is an S - R flip-flop. The block diagram of J - K flip-flop using S - R flip-flop is shown in Fig. 7.37.

The race-around condition can be avoided if $t_p < \Delta t$ as shown in Fig. 7.30(b). The race-around lines can be used in series with the feedback connections in order to increase the loop delay beyond t_p , and hence to prevent the race-around difficulty. The above race-around condition can also be avoided in Master-Slave flip-flops.



Fig. 7.30

A Master-Slave flip-flop is constructed using two separate flip-flops connected serially. The first flip-flop serves as the *master* and the second as a *slave*, and the overall circuit is referred to as a *Master-Slave flip-flop*.

7.10.2 Master-Slave J-K Flip-flop

A Master-Slave flip-flop can be constructed using two J-K flip-flops as shown in Fig. 7.31. The first flip-flop, called the *master*, is driven by the positive edge of the clock pulse; the second flip-flop, called the *slave*, is driven by the negative edge of the clock pulse. Therefore, when the clock input has a positive edge, the master acts according to its J-K inputs, but the slave does not respond since it requires a negative edge at the clock input. When the clock input has a negative edge, the slave flip-flop copies the master outputs. But the master does not respond to the feedback from Q and $\bar{Q}$, since it requires a positive edge at its clock input. Thus, the master-slave flip-flop does not have race-around problem.

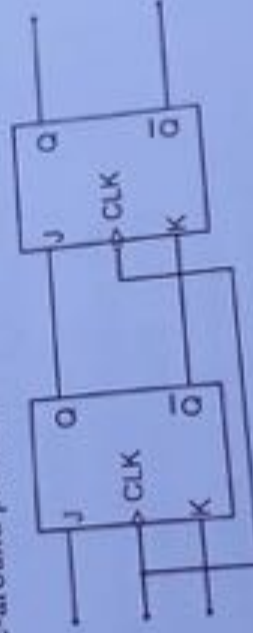


Fig. 7.31 Master-slave J-K flip-flop

A master-slave J-K flip-flop constructed using NAND gates is shown in Fig. 7.32. It consists of two flip-flops connected in series. NAND gates-1 through 4 form the *master* flip-flop and NAND gates-5 through 8 form the *slave* flip-flop. When the clock is positive, a change in J and K inputs cause a change of state in the master flip-flop. During this period, the slave retains its previous state and serves as a buffer between the master and the output. When the clock goes negative, the master flip-flop does not respond, i.e. it maintains its previous state, while the slave flip-flop is enabled and changes its state to that of the master flip-flop. The new state of the slave then becomes the state of the entire master-slave flip-flop. The operation of master-slave J-K flip-flop for different J-K input combinations can be explained as follows:

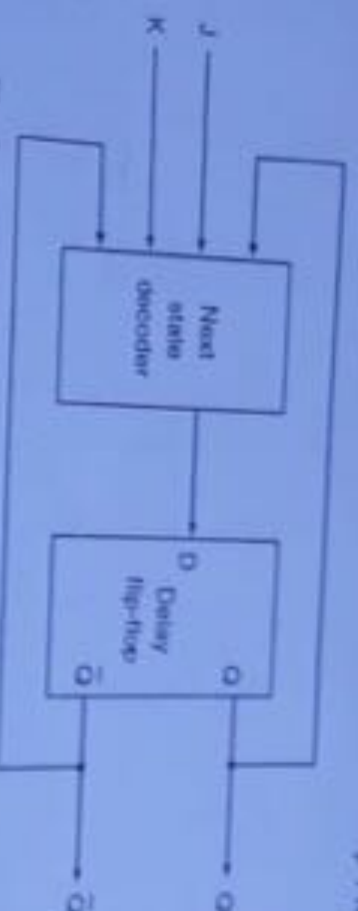


Fig. 7.46 Block diagram of J - K flip-flop using delay flip-flop

Step 1 The operation of J - K flip-flop is well known. Hence, there is no need for word description.

Step 2 The Present State–Next State table for J - K flip-flop can be drawn as shown in Table 7.25.

Table 7.25 PS-NS table for J - K flip-flop

Q_n	J	K	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Step 3 Using the application table of D flip-flop given in Table 7.8, the next state codes, i.e. D value, can be augmented in the above PS-NS table as shown in Table 7.26.

Table 7.26 Excitation table for realisation of J - K flip-flop using D flip-flop

Q_n	J	K	Q_{n+1}	Excitation input D
0	0	0	0	0
0	0	1	0	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1

Step 4 In this step, one can design the next state decoder, i.e. the simplified expression for D , from the excitation map as shown in Fig. 7.47.

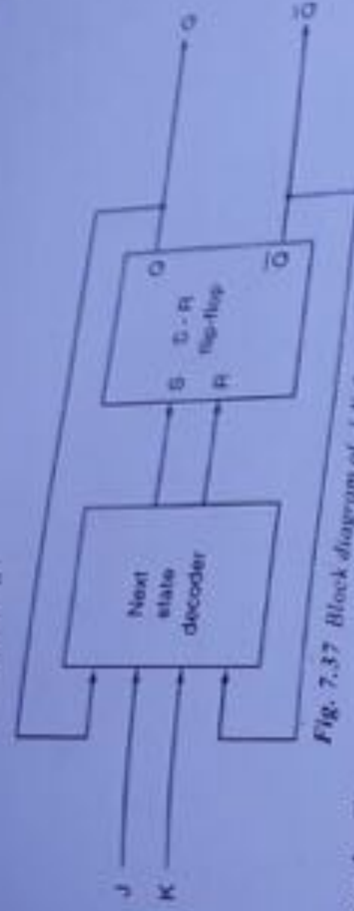


Fig. 7.37 Block diagram of J-K flip-flop using S-R flip-flop

- Step 1** The operation of a J-K flip-flop is well known, hence, description is not given.
- Step 2** The Present State-Next State table for J-K flip-flop can be drawn as shown in Table 7.19.

Table 7.19 PS-NS table for J-K flip-flop

Q_n	J	K	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

- Step 3** Using the application table of S-R flip-flop given in Table 7.4, the next state codes, i.e. S & R values, can be augmented in the above PS-NS table as shown in Table 7.20.

Table 7.20 Excitation table for J-K flip-flop realisation using S-R flip-flop

Q_n	J	K	Q_{n+1}	Excitation inputs	
				S	R
0	0	0	0	0	d
0	0	1	0	0	d
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	d	0
1	0	1	0	0	1
1	1	0	1	d	0
1	1	1	0	0	1

- Step 4** In this step, one can design the next state decoder, i.e. the simplified expressions for S and R, from the excitation maps as shown in Fig. 7.38(a) and (b).

$S = J\bar{Q}_n$

Q_n \ JK	00	01	11	10
0	0	0	1	1
1	d	0	0	d

Step 5

From the above step, $S = J\bar{Q}_n$ and $R = KQ_n$. Now, the circuit for J-K flip-flop using S-R flip-flop can be drawn as shown in Fig. 7.39 with two AND gates in the next state decoder logic.

$R = KQ_n$

Q_n \ JK	00	01	11	10
0	d	d	0	0
1	0	1	1	0

Fig. 7.38(b) Excitation map for R

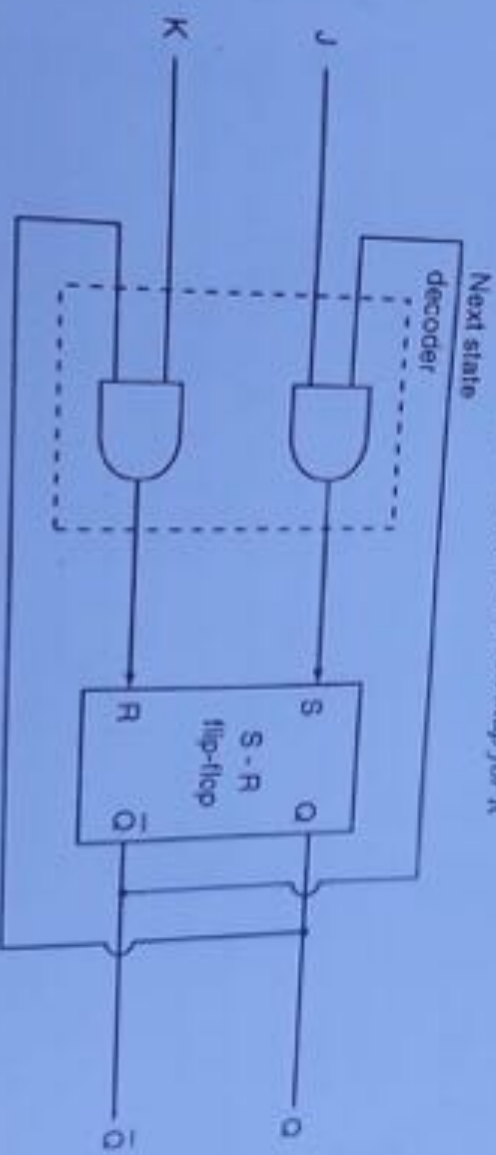


Fig. 7.39 J-K flip-flop using S-R flip-flop

7.11.3 Realisation of Trigger Flip-flop using S-R Flip-flop

Consider the realisation of Trigger flip-flop using S-R flip-flop. Here, the desired flip-flop (X) is Trigger flip-flop and the chosen flip-flop (Y) is S-R flip-flop. The block diagram of Trigger flip-flop using S-R flip-flop is shown in Fig. 7.40.

JK	Q_n			
	00	01	11	10
0	0	0	1	1
1	1	0	0	1

$$D = J\bar{Q}_n + \bar{K}Q_n$$

Fig. 7.47 Excitation map for D

Step 5 From the above step, it can be seen that $D = J\bar{Q}_n + \bar{K}Q_n$. Now, the circuit for $J-K$ flip-flop using D flip-flop is as shown in Fig. 7.48 with two AND gates, one OR gate and one NOT gate in the next state decoder logic.

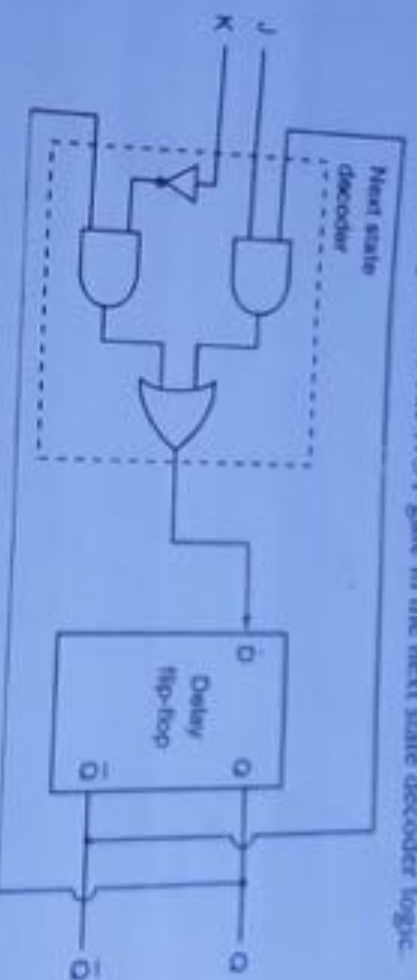


Fig. 7.48 $J-K$ flip-flop using D flip-flop

7.12 APPLICATIONS OF FLIP-FLOPS

Flip-flops find wide applications in counter circuits, frequency dividers, shift and storage registers. A wide variety of serial decoding, comparison and timing functions can be accomplished using flip-flops. Using flip-flops, we can generate a variety of one-shots.

An operation that occurs very frequently in digital system is the transfer of information from one flip-flop or a group of flip-flops to another flip-flop or another group of flip-flops.

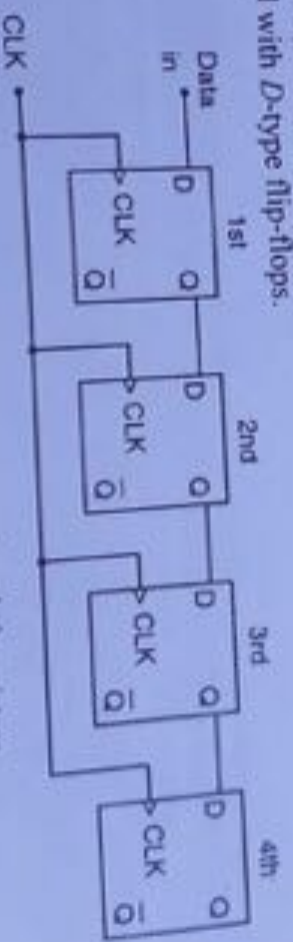
7.12.1 Parallel Data Storage

Several bits of data can be stored simultaneously in a group of flip-flops. This is illustrated in Fig. 7.49. Each of the four parallel data lines is connected to the D input of a flip-flop. The clock inputs of all the flip-flops are connected to a common clock input, so that each flip-flop is triggered at the same time. As positive edge triggered flip-flops are used, the data on the D inputs are stored simultaneously by the flip-flops on the positive edge of the clock. The *clear* inputs are connected to a common $\overline{CLR}$ line, which resets all the flip-flops.

Fig. 7.49 *Flip-flops used for parameter storage*

7.12.2 Shift Registers

Shift registers are used to transfer the contents of one register to another register, or within the same register one bit at a time. Serial shift registers may be implemented using *J-K*, *R-S* and *D-type* flip-flops. Fig. 7.50 shows a serial shift register implemented with *D-type* flip-flops.



7.12.3 Frequency Division

7.12.3 Frequency Division: Flip-flops are also used to divide the frequency of a periodic waveform. When a pulse waveform is applied to the clock input of a J - K flip-flop whose J and K inputs are at 1 state, the Q output is a square wave with one-half the frequency of the clock input. Thus, a single flip-flop is a divide-by-2 device as shown in Fig. 7.51. The flip-flop changes state on each triggering clock edge.

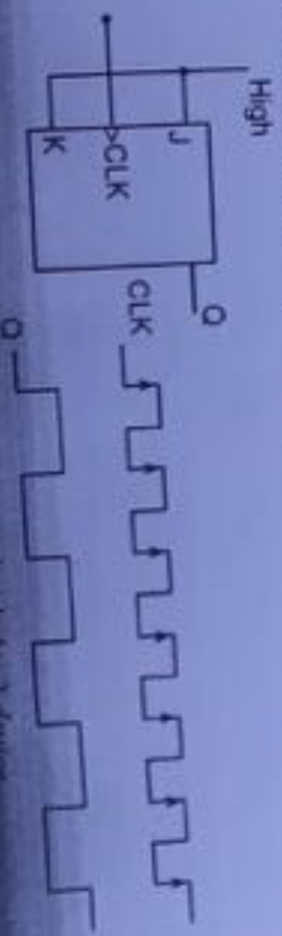


Fig. 7.51 J-K flip-flop as a divide-by-2 device

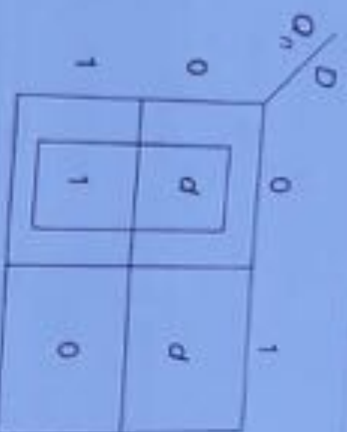
Table 7.24 Excitation table for D flip-flop realisation using J-K flip-flop

Q_n	D	Q_{n+1}	Excitation inputs	
			J	K
0	0	0	0	0
0	1	1	1	0
1	0	0	0	1
1	1	1	0	0

Step 4 In this step, one can design the next state decoder, i.e. simplified expressions for J and K , from the excitation maps as shown in Fig. 7.44(a) and (b).



(a) Excitation map for J



(b) Excitation map for K

Fig. 7.44

Step 5 From the above step, it is seen that $J = D$ and $K = \bar{D}$. Now, the circuit for Delay flip-flop using a J-K flip-flop can be drawn as shown in Fig. 7.45, with a single NOT gate in the next state decoder logic.

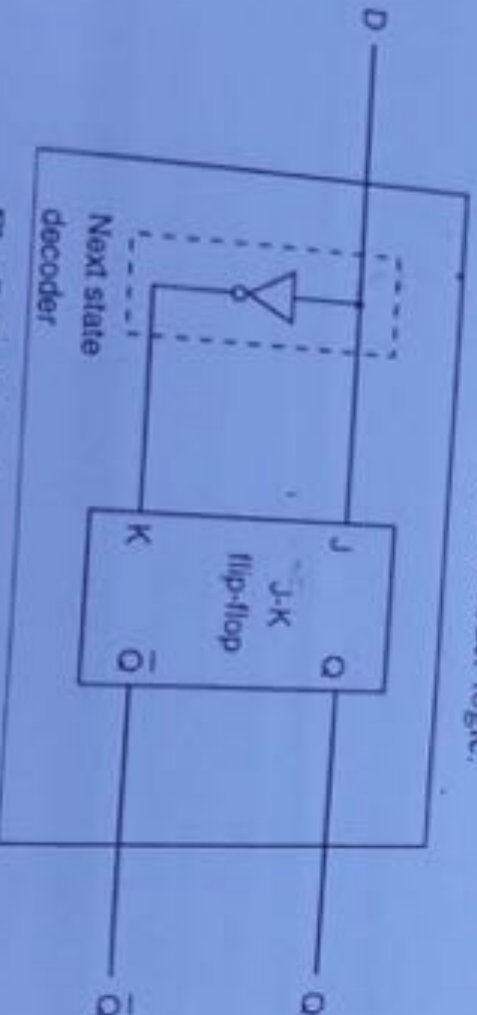


Fig. 7.45 D flip-flop using J-K flip-flop

7.11.5 Realisation of J-K Flip-flop using D Flip-flop

Under this section, the realisation of J-K flip-flop using D flip-flop is taken into consideration. Here, the desired flip-flop (X) is a J-K flip-flop and the chosen one (Y) is a D flip-flop. The block diagram of this realisation is shown in Fig. 7.46.

SR Q _n	SR			
	00	01	11	10
0	0	0	d	1
1	1	0	d	1

Fig. 7.8 Next state (Q_{n+1}) map for S-R latch

From the above K-map,

$$Q_{n+1} = \overline{S}\overline{R} + \overline{R}Q_n$$

$$Q_{n+1} = (S + Q_n)\overline{R}$$

In the above K-map simplification, the don't care combinations are not considered because they correspond to invalid input combinations of S-R latch.

7.3 FLIP-FLOPS

Synchronous circuits change their states only when clock pulses are present. The operation of the basic latch can be modified, by providing an additional control input that determines, when the state of the circuit is to be changed. The latch with the additional control input is called the Flip-Flop. The additional control input is either the clock or enable input.

7.3.1 Types of Flip-Flops

Flip-Flops are of different types depending on how their inputs and clock pulses cause a transition between two states. There are four basic types, namely, S-R, J-K, D and T Flip-Flops.

7.4 S-R FLIP-FLOP

The S-R flip-flop consists of two additional AND gates at the S and R inputs of S-R latch as shown in Fig. 7.9.

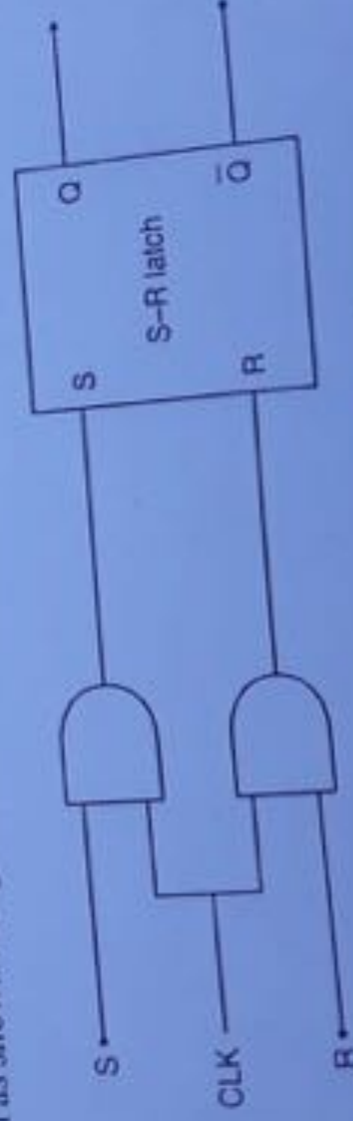


Fig. 7.9 Block diagram of S-R flip-flop

Synchronous Audio

Synchronous combis are used combis
as well as clock data is applied
simultaneously to all of

1. Decides no. of FFs & type of FF
2. Excitation table of FF
3. State diagram & exc. excitation table
4. Boolean expressions

Design a 2 bit synchronous ^{combis} divider
which will output

In synchronous combis, all FFs in
given a clock must be fed out
the necessary flip by using
excitation table.

Synchronous counters.

Synchronous counters are those counter in which clock pulse is applied simultaneously on all FF.

Steps for designing synchronous counter

- 1- Decide no. of FF & type of FF
- 2- Excitation table of FF
- 3- state diagram & Kt excitation table
- 4- Boolean expression

Design a 2 bit synchronous counter with

above all steps.

In synchronous counter, step is given & we need to find out the necessary JKP by using excitation table.

Design 2 Bit Synchronous Counter

no. of FF = no. of bits = 2

Type of FF used is JK FF

Excitation table for

Q ₀	Q ₀	Q ₀	Q ₀
0	0	0	0
0	1	1	1
1	0	0	1
1	1	1	0

State Diagram



Excitation Table

Q ₀	Q ₀	Q ₀	Q ₀
0	0	0	0
0	1	1	1
1	0	0	1
1	1	1	0

could
not
be
can
5/1/15

78 K.
FF

50

$$\begin{array}{c|cc} a_0/a_1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ 1 & 1 & 1 \end{array} \quad \begin{array}{l} k_0 \\ k_1 \end{array}$$

$J_0 = 1$

$$\begin{array}{c|cc} a_0/a_1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ 1 & 1 & 1 \end{array} \quad \begin{array}{l} k_0 \\ k_1 \end{array}$$

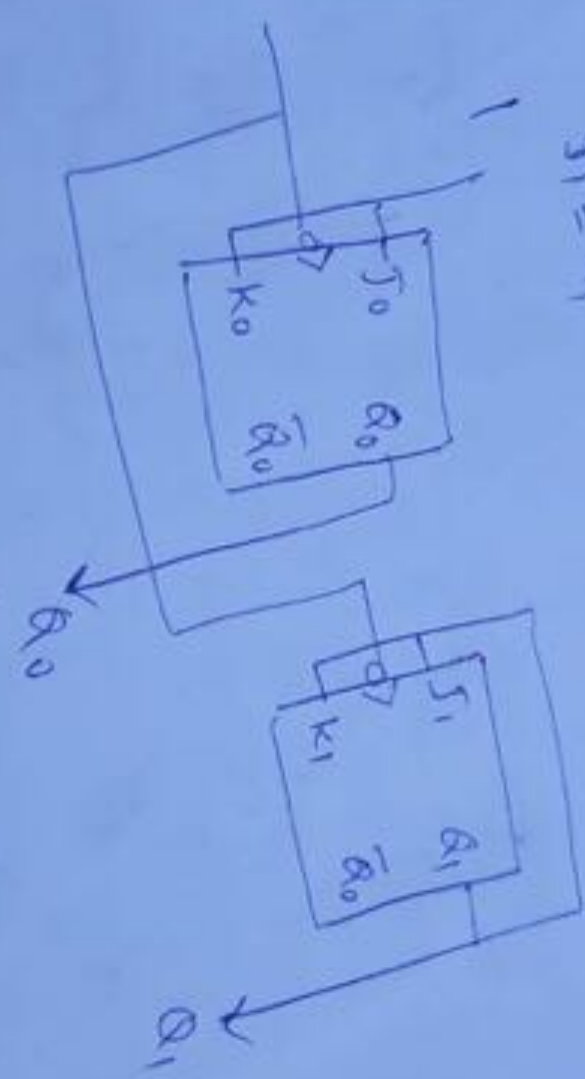
$K_0 = 1$

$$\begin{array}{c|cc} a_0/a_1 & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 1 \end{array} \quad \begin{array}{l} J_1 \\ K_1 \end{array}$$

$J_1 = a_1$

$$\begin{array}{c|cc} a_0/a_1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ 1 & 1 & 1 \end{array} \quad \begin{array}{l} J_1 \\ K_1 \end{array}$$

$K_1 = a_1$



Up Down Counter

Steps involved in ~~2 bit~~ Synchronous Counter.

1. Decide No. of FF & Type
2. Excitation Table of FF used.
3. State Diagram & K_{ex} excitation Table.
4. Boolean expression

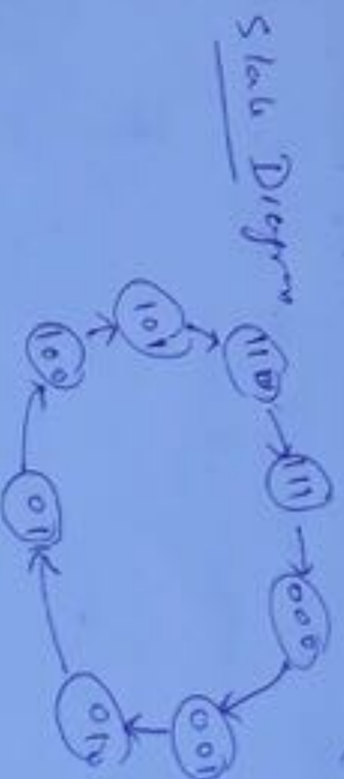
Design 3 bit Synchronous Counter
 o/p w/ give us need to design 57P.

1. No. of FF = No. of bits = 3.
 we are using J & K FF.

2. Excitation Table for J & K FF.

Q _n	Q _{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

3. State Diagram: Total states = 8.
 No. of max. count = $2^m - 1$
 = $2^3 - 1 = 7$
 (0-7)



Ext. Excitable

$Q_0 \backslash A_1 A_0$ J_0

0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	1	0	1	1	0
1	1	0	1	1	1	0

$J_0 = 1$

	$P.S.$	$A_2 A_1 A_0$	$N.S.$	$A_2 A_1 A_0$	J_0	K_0	J_1	K_1	J_2	K_2
1	000	000	010	0	1	0	1	0	0	0
2	001	010	011	1	0	1	0	0	0	0
3	010	100	101	1	0	0	1	0	0	0
4	011	101	110	1	0	0	1	0	0	0
5	100	110	111	1	0	0	1	0	0	0
6	101	111	000	1	0	0	1	0	0	0
7	110	000								

$Q_0 \backslash A_1 A_0$ K_0

0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	1	0	1	1	0
1	1	0	1	1	1	0

$K_0 = 1$

$Q_0 \backslash A_1 A_0$ J_1

0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	1	0	1	1	0
1	1	0	1	1	1	0

$J_1 = 0$

$Q_0 \backslash A_1 A_0$ K_1

0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	1	0	1	1	0
1	1	0	1	1	1	0

$K_1 = 0$

$$J_0 = Q_1 Q_0$$

Q_1	Q_0	J_0
0	0	0
0	1	0
1	0	1
1	1	0

$$J_0 = Q_1 Q_0$$

$$K_2 = Q_1 Q_0$$

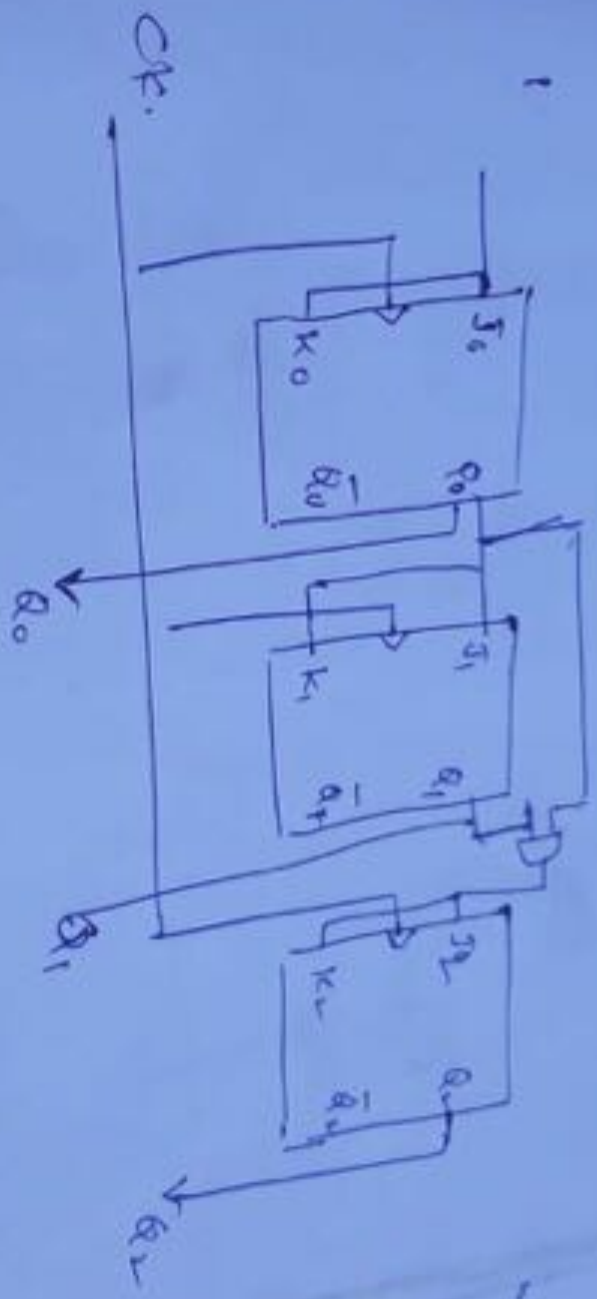
Q_1	Q_0	K_2
0	0	0
0	1	0
1	0	1
1	1	0

$$K_2 = Q_1 Q_0$$

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = Q_0$$

$$J_2 = K_2 = Q_1 Q_0$$



Q_2
 Q_1
 Q_0

2 bit up/down synchronous counter
 Total no. of states $= 2^n = 4$
 more count = 4+1 = 5

Let if up count $T=0$

Down count $T=1$



Truth Table

Q_1	Q_0	T
0	0	0
0	1	0
1	0	1
1	1	0

Q_1	Q_0	T
0	0	0
0	1	0
1	0	1
1	1	0

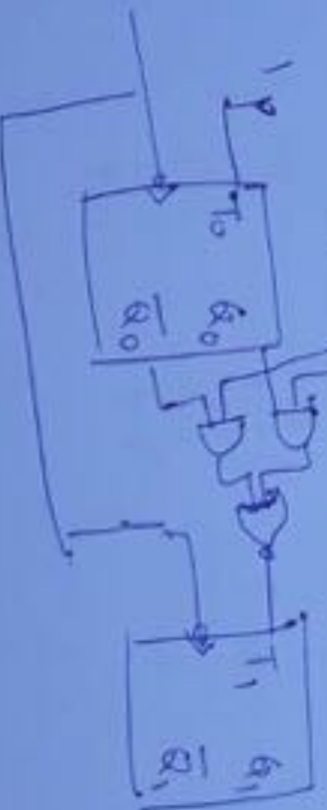
Truth Table

Q_1	Q_0	T
0	0	0
0	1	0
1	0	1
1	1	0

Truth Table

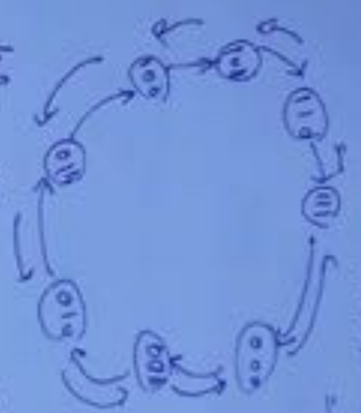
Q_1	Q_0	T
0	0	0
0	1	0
1	0	1
1	1	0

$$T_1 = \bar{m}Q_0 + m\bar{Q}_0$$



3 Bit up/down Synchronous Counter

Table No. of states = 8
max count = 8-1



0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

let say $m=0$ if $m=0$ then up counting
if $m=1$ then down counting

m	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

T_2	T_1	T_0
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Excitation Table of T	
Q_n	Q_{n+1}
0	0
0	1
1	0
1	1

MARKS
 Question marks (like answers) are in
 red. My comments are in
 blue. The student's name is
 The University of
 To submit

$$T_0 = \bar{m}a_0a_0 + m\bar{a}_0a_0$$

a_0	0	1
m	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1

$$T_2 = \bar{m}a_0a_0 + m\bar{a}_0a_0$$

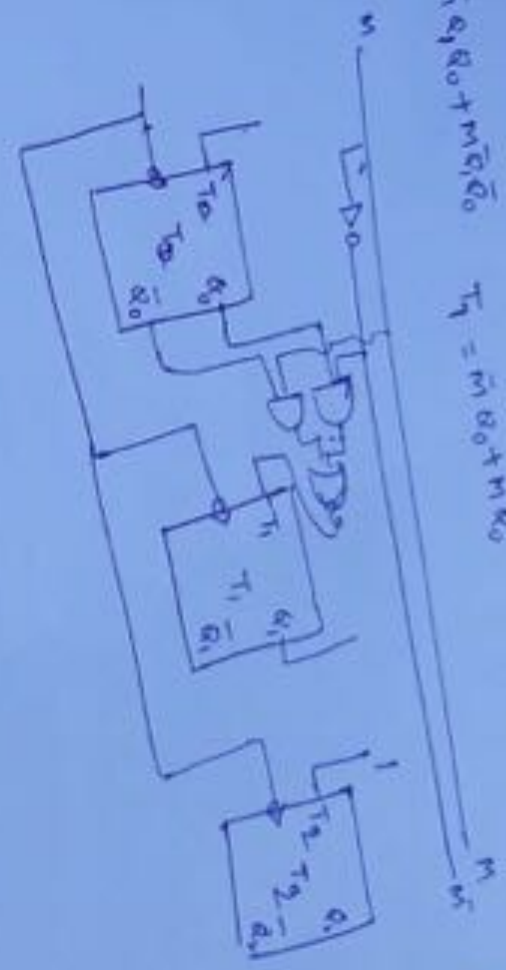
$$T_1 = \bar{m}a_0 + m\bar{a}_0$$

a_0	0	1
m	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1

$$T_1 = \bar{m}a_0 + m\bar{a}_0$$

$$T_0 = 0$$

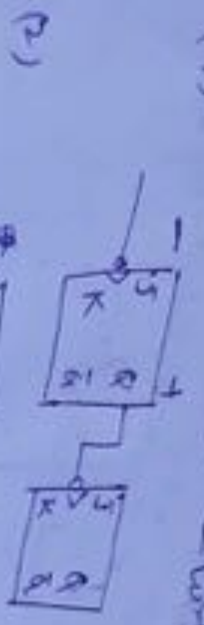
a_0	0	1
m	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1
a_0	0	1



when we
 it is
 the
 it
 on door

Up Down Counter

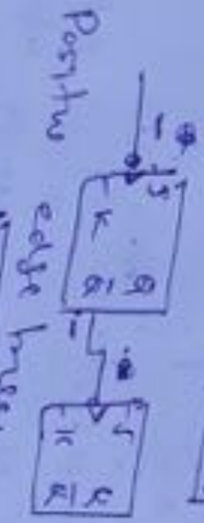
(1)



Up Count: $(-Q + \text{Combinational})$

(Carry out)

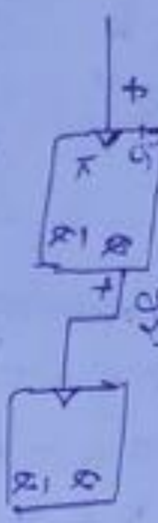
(2)



Down Count: $(-Q -)$

Sum Sign $(+2+V)$

(3)

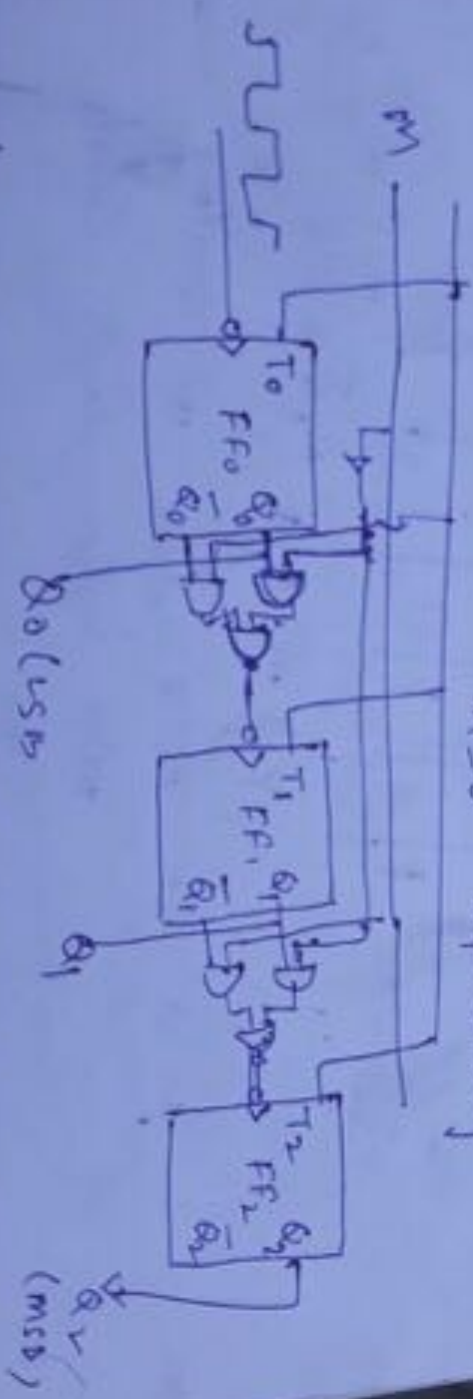


Up Count $(+2-)$



2 bit Asynchronous Up Down Counter

we use M control for up & down
when $M=0$ up counting
when $M=1$ down counting



when we count preceding FF with previous Q/P (Q)
then it works as up count & why?
It is connected with $\bar{Q}$ then it will act
as down count.

up 8 can be

MOD-6

Ripple

(only 6 up)

7-1=0 No. of state = 6

Max count = $2^n - 1$

= $6 - 1 \Rightarrow 5$



No. of FF = 3

As per bit Told no. of state will be 8-

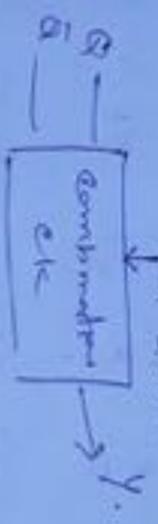
max count = $8 - 1$
= 7

Dec	Q ₂	Q ₁	Q ₀	clean
0	0	0	0	1
1	0	0	1	1
2	0	1	0	1
3	0	1	1	1
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	X

at should be zero after 5.

we are preset & clear; .. preset & clear is also called direct or overriding IP. preset makes up value 0 clear makes up value 1 in normal mode in clear is high

To derive next FF, we need to make a combination etc. m chosen select next up & down



Logic and Expression for next

Possible for up counting

m	Q	Q-bar	Y
0	0	1	0
1	0	0	1
2	1	0	0
3	1	1	1
4	0	1	0
5	0	0	1
6	1	1	0
7	1	0	1

Possible for down counting

m	Q	Q-bar	Y
0	0	1	0
1	0	0	1
2	1	0	0
3	1	1	1
4	0	1	0
5	0	0	1
6	1	1	0
7	1	0	1

Y will depend on Q for up counting.
 Q for up counting.
 & m=0 for up counting
 Y will depend on Q for down counting.
 Q m=1 for down counting

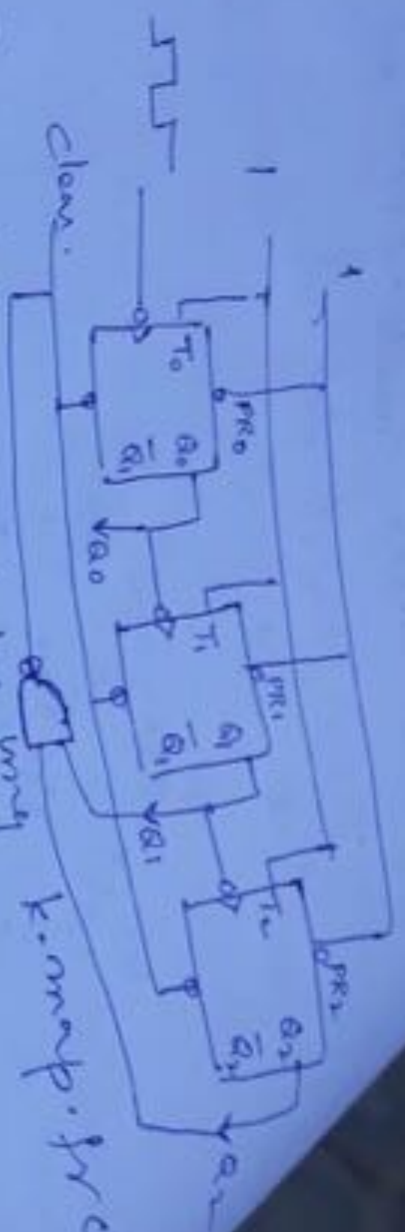


$$Y = \bar{m}_1 \cdot Q + m_1 \cdot \bar{Q}$$

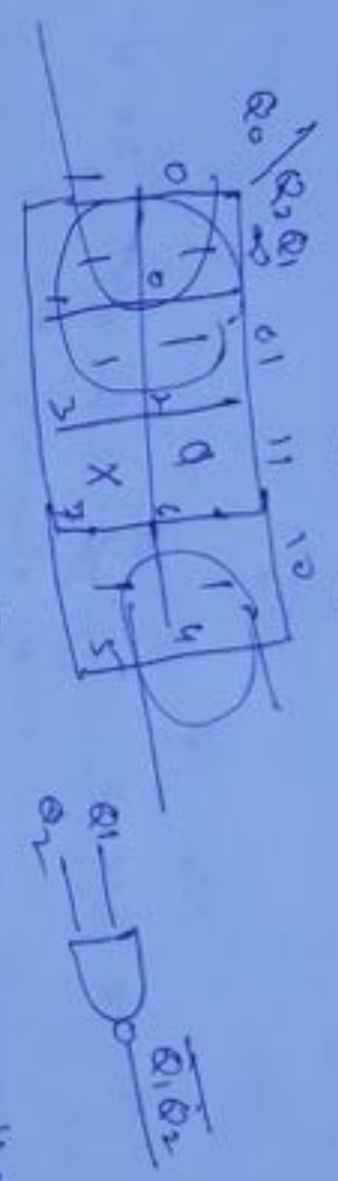
Also let m=0 for up counting
 m=1 for down counting



10/11/2020

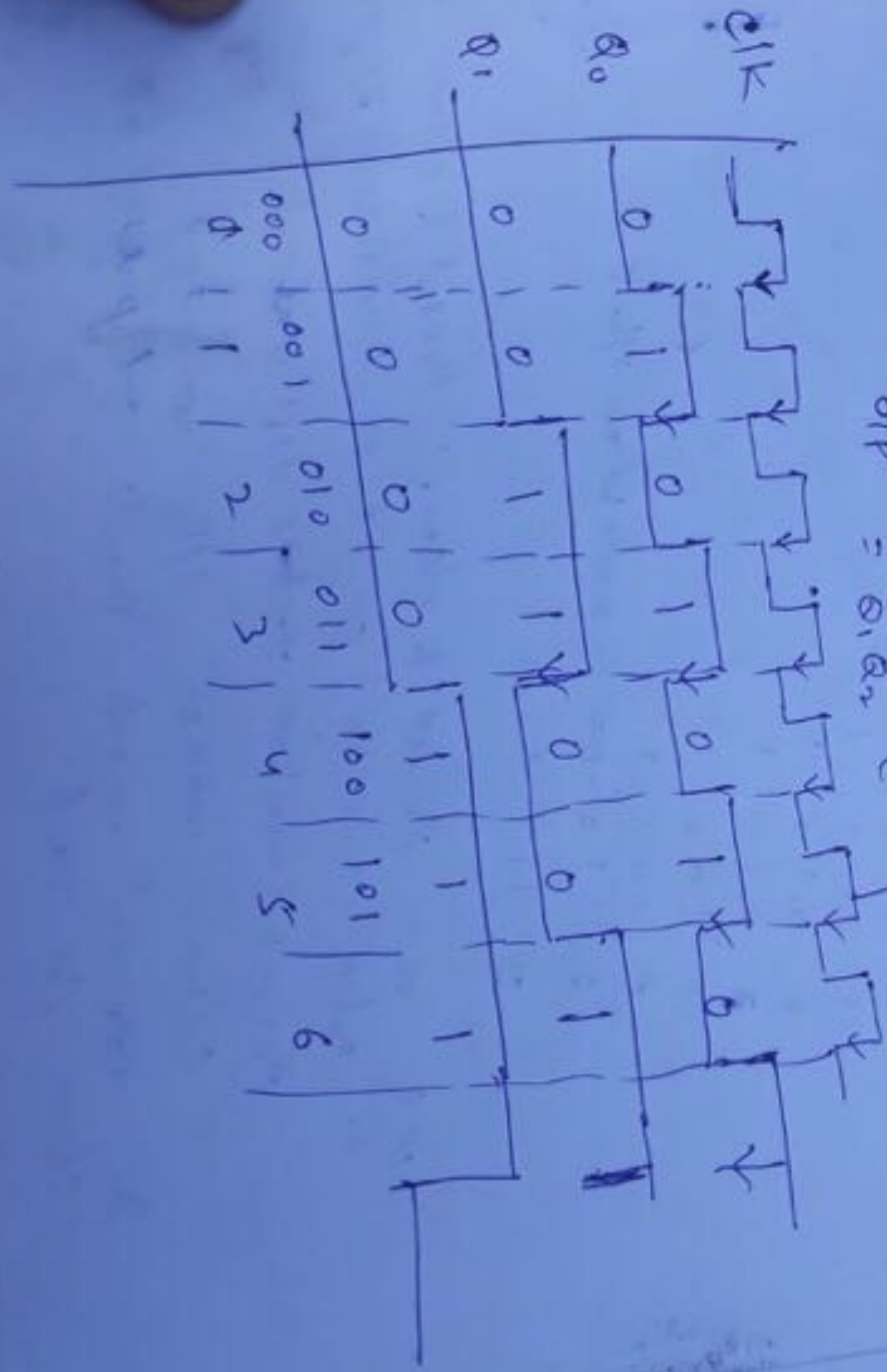


Combination of 3 bits by using J-K map for clear.



$$clear = \overline{Q_2} + \overline{Q_1}$$

o/p = $\overline{Q_1} \overline{Q_2}$ (using demorgan's theorem)



Decade Counter / MOD 10 Ripple Counter

94 means mean 10

Total 10. if $6 \pmod{10}$ count

Max. count = $2^n - 1$

$$= 10 - 1 \Rightarrow 9$$

94 will be $(0-9) \Rightarrow (0-1001)$

No. of FF = No. of bits

$$= 4$$

With 4 bits, we will get 16 states

& max count will be (0-15)

So we need to stop count at 9

To stop count at 9

we use Reset &

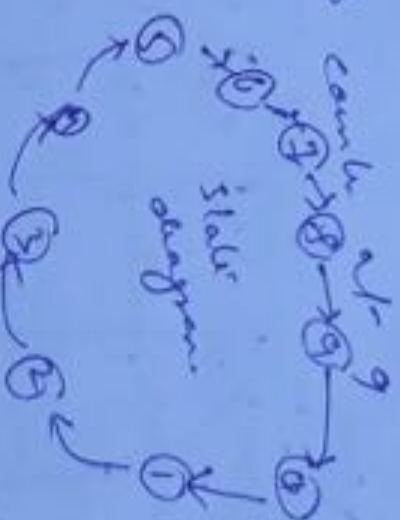
clearst concept, this

are called overriding

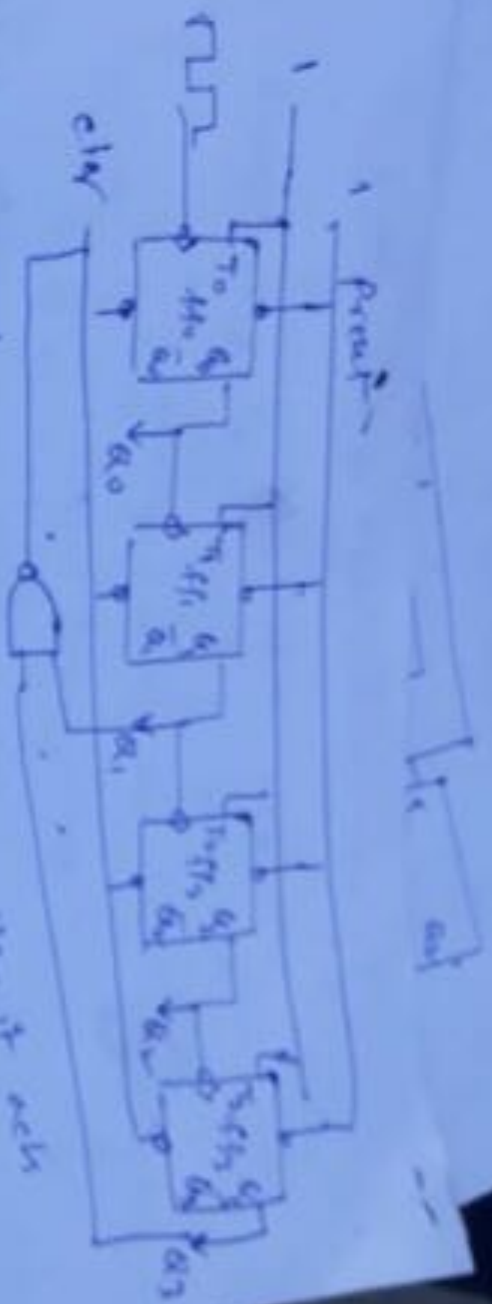
& or direct FF.

i.e. whatever input we have at FF,

this FF will be consider.



Method 1 is bit expensive



signaling Reset & clear to 1, then it acts as normal mode.

for T0 active then we need to make their value zero.



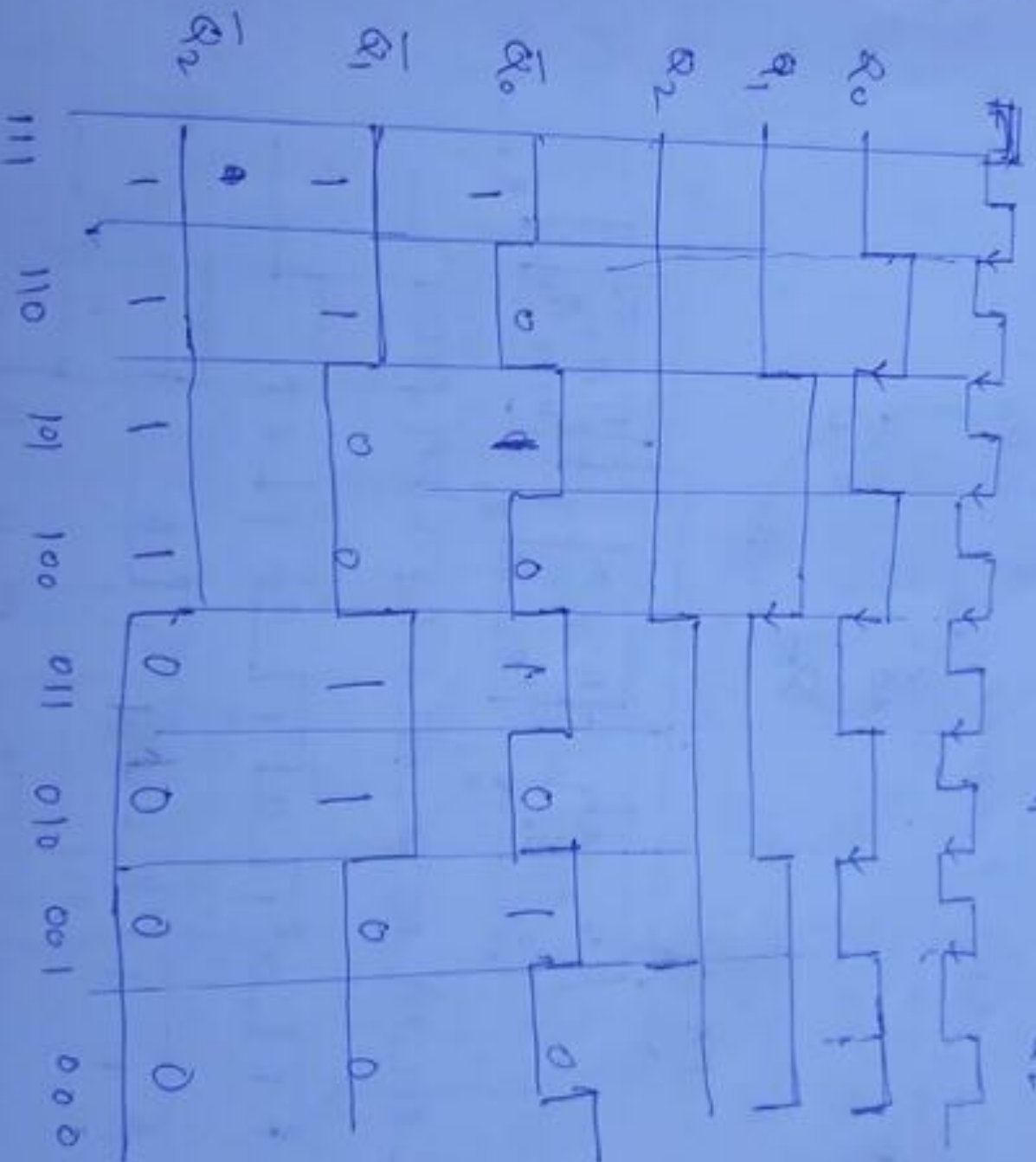
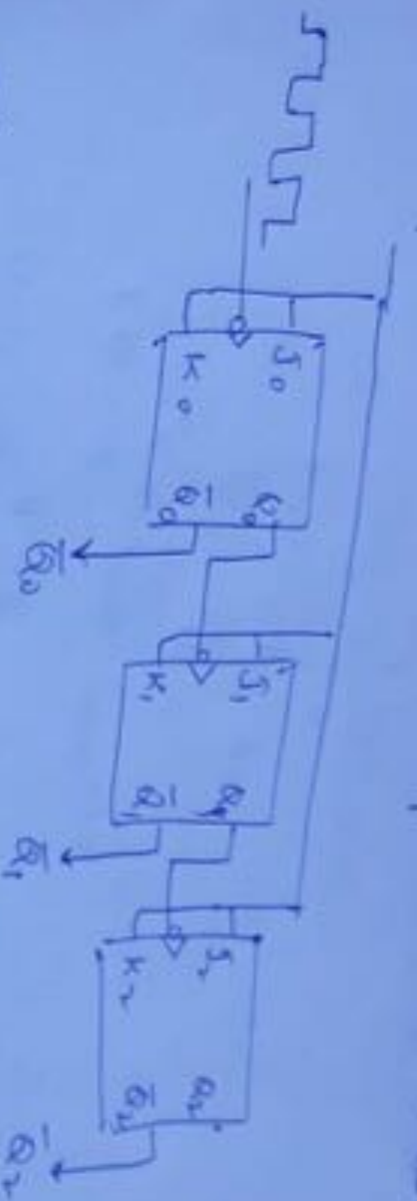
$$clear = \overline{a_3 + a_1} = \overline{a_3 \cdot a_1}$$

a3	a2	a1	a0	clear
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Method I

3 bit

Asynchronous Down Counter



Method-2

3 bit Asynchronous Counter Down

(two edge trigger)

$$MOD\ N = 2^m$$

$$= 2^3$$

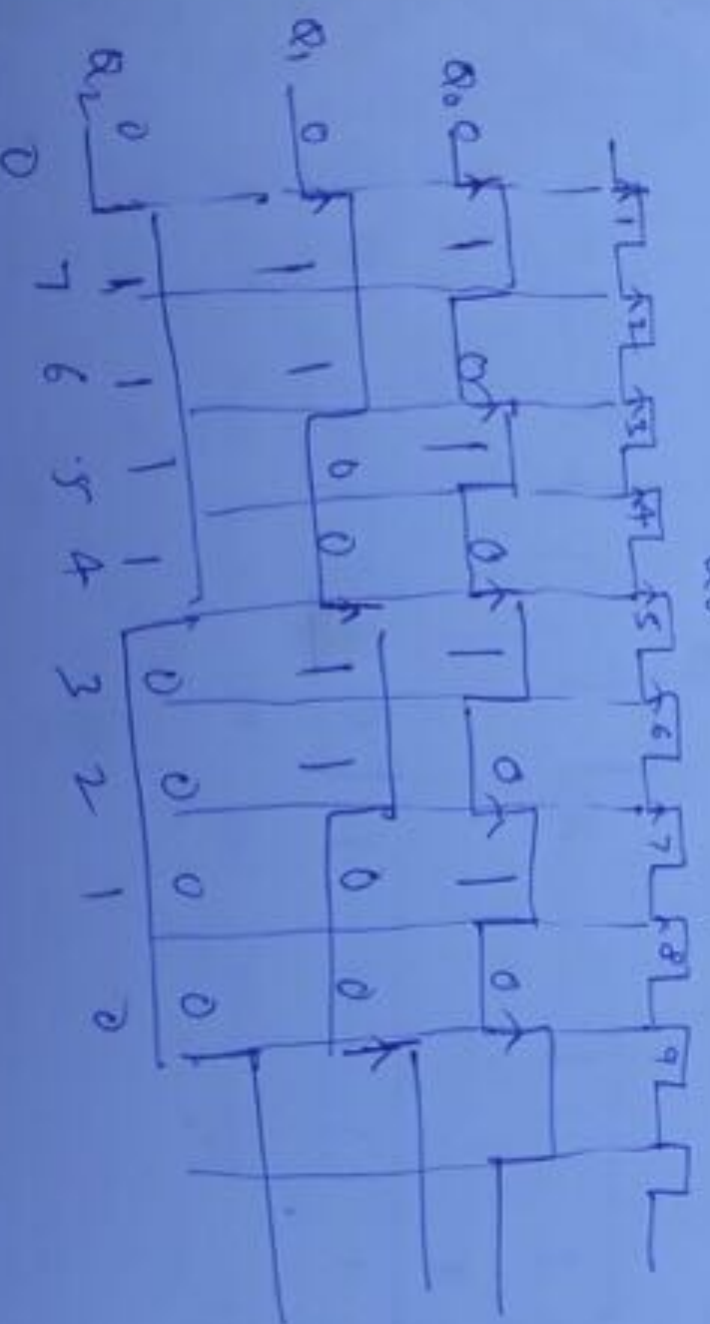
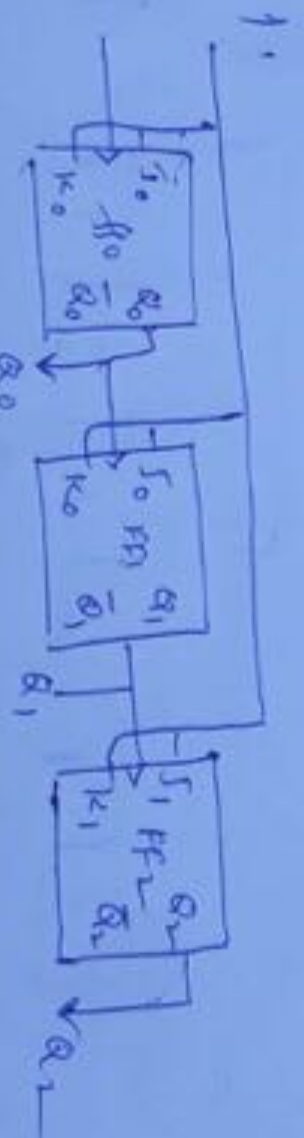
$$MOD = 8\ \text{Counts}$$

$$No. of\ states = 8$$

$$No. of\ max\ counts = 2^m - 1$$

$$= 8 - 1 \Rightarrow 7$$

State Diagram



Method 3

Design a 3 bit down counter (sequence in descending order)

Mod Number (M) = 2^n

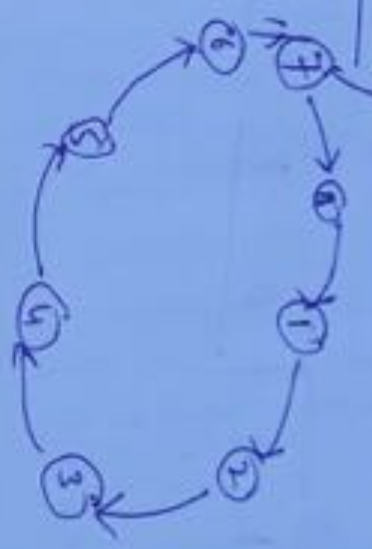
= $2^3 \Rightarrow 8$

No. of states = 8

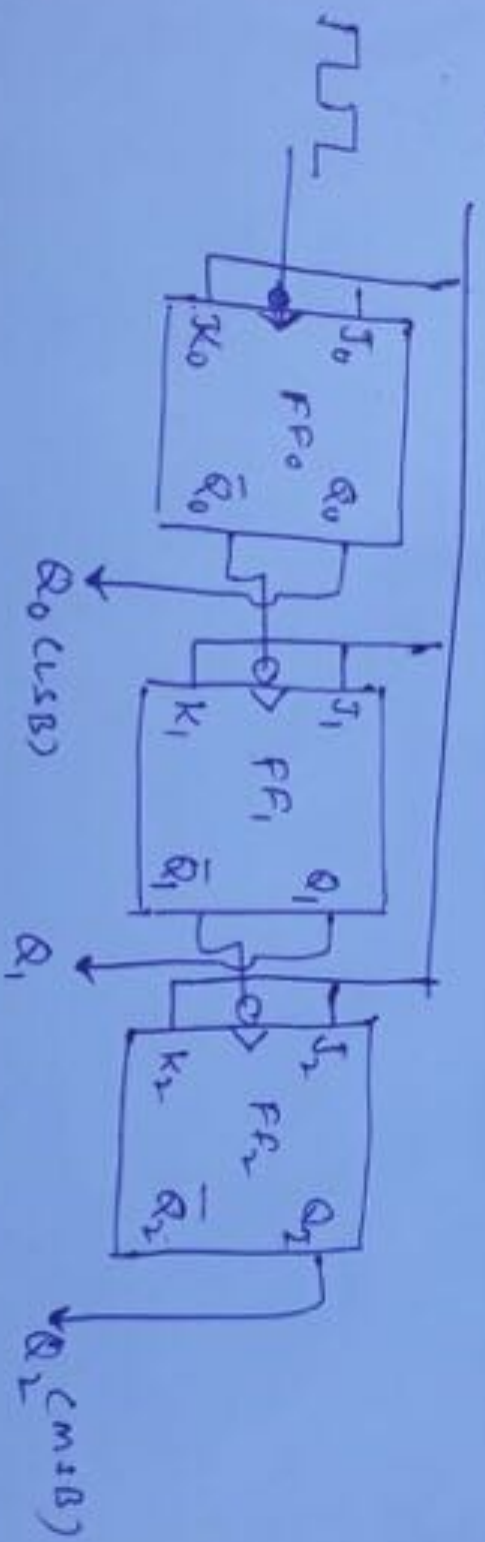
No. of up count = 2^{M-1}

= $2^{3-1} \Rightarrow 7$

State Diagram



Note:
 ++ down counter
 +- up counter



Truth Table

up count

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Down Counter

state

7	1	1	1
6	1	1	0
5	1	0	1
4	1	0	0
3	0	1	1
2	0	1	0
1	0	0	1
0	0	0	0

Timing Diagram

